EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Week 9: Memory and Persistence

Saving task progress and using memory across conversations

Instructor: Arash Saifhashemi

What you will learn

What should an agent remember, how should it use that information, and when should it change or forget it?

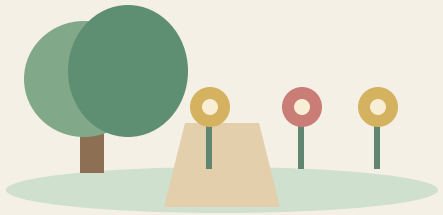
- **Context and state:** distinguish one model call's input from working task data.
- **Recovery:** trace how checkpoints save progress and resume work.
- **Writing memory:** save confirmed facts with their source, scope, and lifetime.
- **Retrieval:** choose valid memories that help with a later request.
- **Maintenance:** correct facts, merge duplicates, expire exceptions, and delete records.
- **Evaluation:** test recovery, recall, corrections, and record removal.

We manage saved application data. Model training is outside the scope.

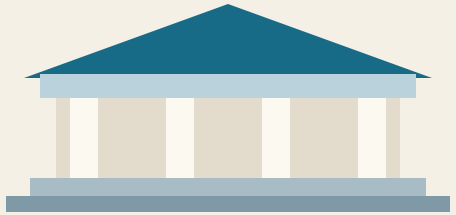
Running example: a weekend planner

An assistant finds activities, checks cost and transit,
and drafts an afternoon itinerary.

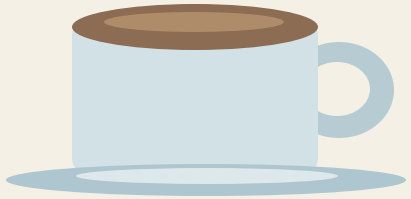
> *“Plan Saturday afternoon in LA. Keep it under $50. I will not have a car.”*



Garden A



Museum B



Cafe C

Why the assistant needs memory

- **After a server crash:** resume from saved options instead of repeating the search.
- **Next weekend:** recall past visits and confirmed preferences instead of asking again.

Why do agents need memory?

LLMs are stateless: Every model call starts from zero and sees only its own prompt.

The model doesn't remember anything. Your application does.

Problem 1: the server crashes

- Without saved progress, the planner repeats every search.
- Fix: **checkpoints** (short-term memory).

Problem 2: next weekend

- It asks about the car again and suggests the crowded cafe
- Fix: **long-term** memory.



Why do agents need memory?

LLMs are stateless: Every model call starts from zero and sees only its own prompt.

The model doesn't remember anything. Your application does.

Problem 1: the server crashes

- Without saved progress, the planner repeats every search.
- Fix: **checkpoints** (short-term memory).

Problem 2: next weekend

- It asks about the car again and suggests the crowded cafe
- Fix: **long-term** memory.



Other reasons agents need memory:

1. The context window fills up on long tasks.
2. Learning from feedback instead of repeating mistakes.
3. Pausing for human approval, then resuming.
4. Lower cost than re-sending the full history.
5. Staying consistent across sessions.

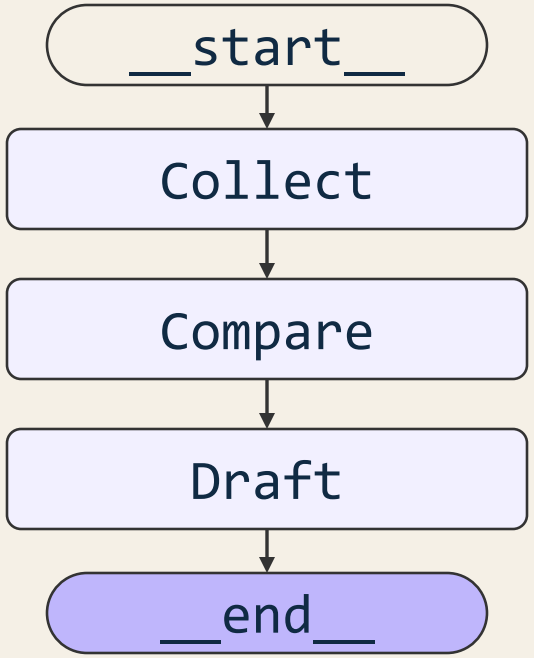
Context and state

What does one model call need, and what must the task keep between steps?

Collecting, comparing, and drafting require data to pass between steps.

Weekend planner: LangGraph workflow

A node is a Python function that reads state and returns an update.



Node functions

- **Collect node:** Retrieve venue details. For Garden A and Cafe C.
- **Compare node:** Check budget and travel. Choose using the venue facts.
- **Draft node:** Write the outing plan.

State carries the request, venue facts, chosen activity, and draft between nodes.

Nodes read state and return updates: LangGraph Graph API.

Context, state, checkpoints, and memory

After the Collect node, the candidate activities can serve several roles.

Kind	Purpose	Weekend planner example
Model context	Sent to one model call	Request + three candidates

Context is selected for one call. State carries working data to the next node.

LangGraph Persistence: roles of state, checkpoints, and long-term memory.

Context, state, checkpoints, and memory

After the Collect node, the candidate activities can serve several roles.

Kind	Purpose	Weekend planner example
Model context	Sent to one model call	Request + three candidates
Application state	Working data between nodes	Candidates saved; chosen is still empty

Context is selected for one call. State carries working data to the next node.

LangGraph Persistence: roles of state, checkpoints, and long-term memory.

Context, state, checkpoints, and memory

After the Collect node, the candidate activities can serve several roles.

Kind	Purpose	Weekend planner example
Model context	Sent to one model call	Request + three candidates
Application state	Working data between nodes	Candidates saved; chosen is still empty
Execution checkpoint	Saved state + pending work	Candidates saved; `Compare` runs next

A checkpoint saves the working data and pending work needed to resume this task.

LangGraph Persistence: roles of state, checkpoints, and long-term memory.

Context, state, checkpoints, and memory

Kind	Purpose	Weekend planner example
Model context	Sent to one model call	Request + three candidates
Application state	Working data between nodes	Candidates saved; chosen is still empty
Execution checkpoint	Saved state + pending work	Candidates saved; `Compare` runs next
Long-term memory	Confirmed fact for future tasks	*“I usually take public transit”*

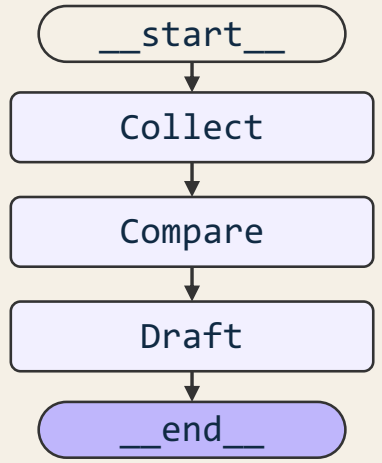
The checkpoint resumes this Saturday’s task. Long-term memory guides a new weekend plan.

LangGraph Persistence: roles of state, checkpoints, and long-term memory.

LangGraph State

User request:

*"Help me choose between **Garden A** and **Cafe C** for Saturday. My budget for the outing is **$50**, and I **won't have a car**."*



Schema: allowed state fields

```python
from typing_extensions import TypedDict

class WeekendState(TypedDict):
    request: str
    candidates: list[str]
    venue_facts: list[str]
    chosen: list[str]
    draft: str
```

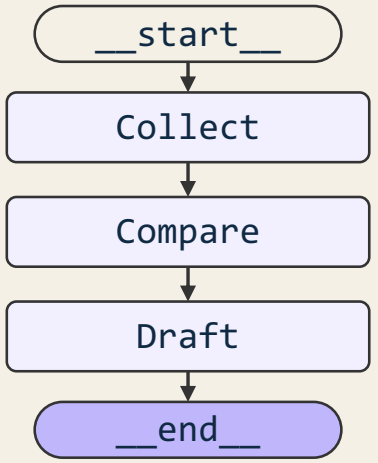
Values before retrieval

```python
state = {
    "request": user_request,
    "candidates": ["Garden A",
                   "Cafe C"],
    "venue_facts": [],
    "chosen": [],
    "draft": ""
}
```

The user supplied the names. Venue facts have not been retrieved yet.

Raw task data in state: Thinking in LangGraph, step 3.

State after the Collect node



The Collect node retrieves venue documents and returns a state update.

```
def collect(state):
    facts = venue_search(state["candidates"])
    return {"venue_facts": facts}
```

`venue_search` looks up venue documents and returns a list of text snippets.

State after applying the update

```
state = {
    "request": user_request,
    "candidates": ["Garden A", "Cafe C"],
    "venue_facts": facts,
    "chosen": [],
    "draft": ""
}
```

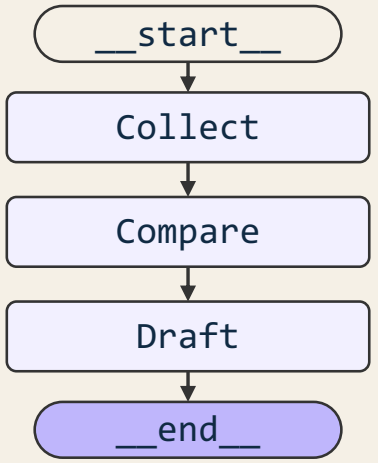
Example retrieved snippets

```
facts = [
    "Garden A listing: open Sat; "
    "entry $12; return bus $6.",
    "Cafe C listing: open Sat; "
    "meal $25; return bus $6."
]
```

chosen and draft are still empty.

Raw search results and partial state updates: Thinking in LangGraph, steps 3–4.

Building the model's input from state



The Compare node uses retrieved facts for retrieval-augmented generation (RAG).

chooser is a chat model configured to return a dictionary with a chosen list.

```python
import json

def compare(state):
    data = {
        "request": state["request"],
        "candidates": state["candidates"],
        "venue_facts": state["venue_facts"],
    }
    messages = [
        ("system", "Choose using the request and facts."),
        ("human", json.dumps(data)),
    ]
    choice = chooser.invoke(messages)
    return {"chosen": choice["chosen"]}
```

Read selected fields from state.

Add instructions. Keep facts as data.

Send messages to the model as context.

Return chosen as a state update.

Build prompts inside nodes from state: Thinking in LangGraph, steps 3–4.

State and recovery in LangGraph

How do we save the weekend planner’s state, inspect it, and resume after a crash?

Use working Python examples to save task data, inspect saved progress, and resume interrupted work.

Building the weekend planner graph

The builder registers the Python node functions and sets their execution order.

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(WeekendState)
builder.add_node("Collect", collect)
builder.add_node("Compare", compare)
builder.add_node("Draft", draft)
builder.add_edge(START, "Collect")
builder.add_edge("Collect", "Compare")
builder.add_edge("Compare", "Draft")
builder.add_edge("Draft", END)
```

Collect node update

```
def collect(state):
    facts = venue_search(
        state["candidates"])
    return {
        "venue_facts": facts
    }
```

The builder describes nodes and their execution order. Compilation creates an executable graph.

LangGraph Graph API: StateGraph, add_node, add_edge, and compile (adapted).

Compiling with a LangGraph checkpointer

A checkpointer saves and loads snapshots of graph state. Attach it when compiling the executable graph.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = builder.compile(
    checkpointer=checkpointer
)
```

Create a saver

InMemorySaver keeps snapshots in RAM while this process runs.

Attach it at compile

The executable graph uses this saver to save and load state.

Changing the saver type changes storage without changing the node functions. Other savers include SqliteSaver and PostgresSaver.

LangGraph: compile with a checkpointer to save and load snapshots.

LangGraph threads keep separate task histories

Thread: a saved workflow history selected by `thread_id`.
Several calls can continue one thread.

```
saturday = {"configurable": {
    "thread_id": "saturday-1"
}}
next_weekend = {"configurable": {
    "thread_id": "next-weekend"
}}
graph.invoke(first_input, saturday)
graph.invoke(next_input, next_weekend)
snapshot = graph.get_state(saturday)
```

Saturday's history

first_input: Saturday, under $50, no car.

A separate history

next_input: Sunday, under $70, car available.

Read Saturday's state

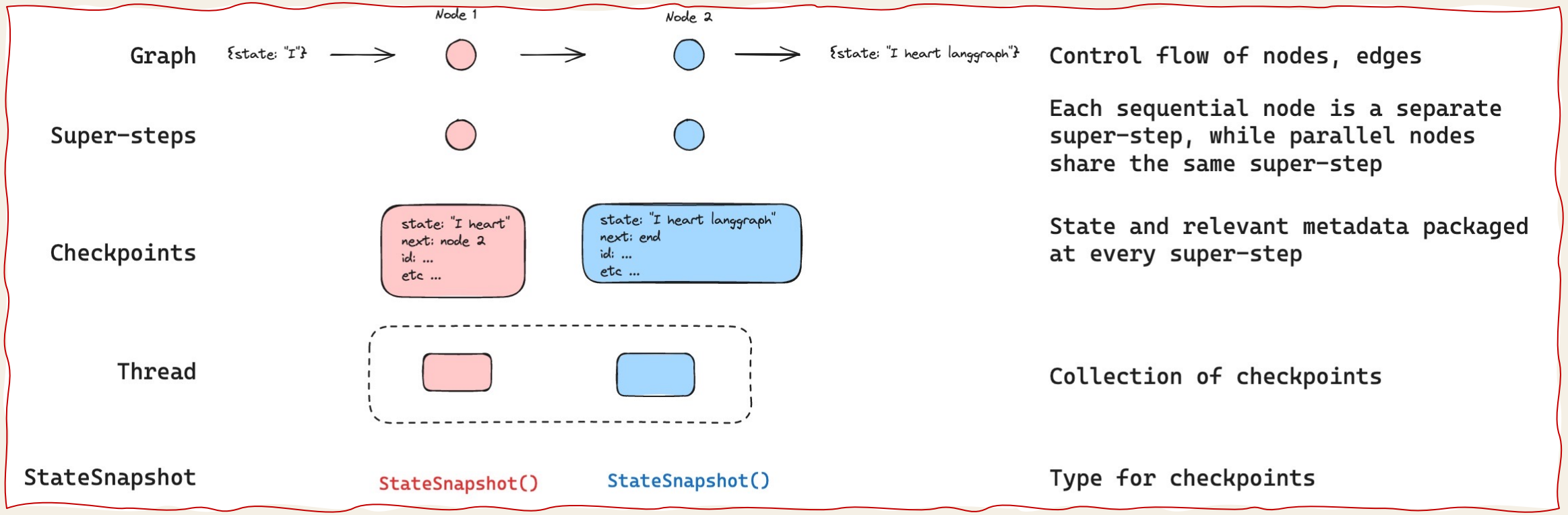
Read the values saved for saturday-1.

Reuse the thread ID to continue a task. Use a new ID for a separate task.

LangGraph Checkpointers: thread_id selects saved state and execution history.

LangGraph checkpoints and threads

A superstep is one execution round. A thread groups its saved checkpoints. `StateSnapshot` is the Python object representing one checkpoint.



LangGraph original figure: execution, checkpoints, thread, and snapshots.

Saving checkpoints in a SQLite file

Another saver like SqliteSaver can write checkpoints to a local database file.

```
from langgraph.checkpoint.sqlite import SqliteSaver
with SqliteSaver.from_conn_string(
    "weekend.sqlite"
) as saver:
    graph = builder.compile(checkpointer=saver)
    config = {"configurable": {
        "thread_id": "saturday-1"
    }}
    graph.invoke(initial_state, config,
                 durability="sync")
```

SQLite file

weekend.sqlite holds the saved checkpoints on disk.

Select this thread

saturday-1 selects this task's saved history.

Wait for saving

Wait for each save before the next execution round.

durability is a parameter passed to invoke to specify when/how the checkpointer saves the state

LangGraph: SQLite checkpointer, thread configuration, and sync durability (adapted).

Reading the saved LangGraph checkpoint

`graph.get_state(config)` returns a `StateSnapshot`: saved `values` and details about what runs `next`.

```
snapshot = graph.get_state(config)
print(snapshot.values)
print(snapshot.next)
```

Read the saved snapshot

- **config:** thread and checkpoint IDs
- **values:** data saved after the `Collect` node
- **next:** ("Compare",), the pending node
- **tasks:** scheduled work and errors

Saved fields (excerpt) and next

```
{"candidates": ["Garden A", "Cafe C"],
 "chosen": [], "draft": ""}
("Compare",)
```

LangGraph Checkpointers: get_state and StateSnapshot fields.

LangGraph: recovering a run after a crash

The **`Collect`** node's output is saved. The process stops before the `Compare` node finishes.

```
with SqliteSaver.from_conn_string(
    "weekend.sqlite"
) as saver:
    graph = builder.compile(checkpointer=saver)
    config = {"configurable": {
        "thread_id": "saturday-1"
    }}
    result = graph.invoke(None, config,
                          durability="sync")
```

Reopen the same file

Saved candidates remain in weekend.sqlite.

Select the same thread

Load Saturday's state and pending work.

Resume saved work

None resumes the pending `Compare` node.

The `Compare` node resumes, then the `Draft` node runs. The completed `Collect` node does not repeat.

LangGraph: thread-based checkpoint loading and invoke(None) resumption.

Writing and correcting long-term memory

The outing is over. What should the assistant save for future weekends?

Separate what happened from an inference. Save confirmed information with its source and scope.

Working memory and long-term memory

In the weekend planner, LangGraph state holds working memory across steps.

- **Working memory: active task data**; each call receives selected information.
  - **Weekend planner**: this Saturday's request, budget, and activity candidates.

Long-term memory: three kinds of stored information

- **Episodic:** records of particular past experiences.
  Example: the previous outing's places, actions, and feedback.
- **Semantic:** facts and generalizations.
  Example: the user usually prefers public transit.
- **Procedural:** know-how in agent code and learned model behavior.
  Example: a reusable function that checks cost and transport constraints.

Reflection produces insights that can be stored in semantic memory.

Memory types / reflection: CoALA §§4.1, 4.5. State: LangGraph Graph API.

CoALA: how memory supports decisions

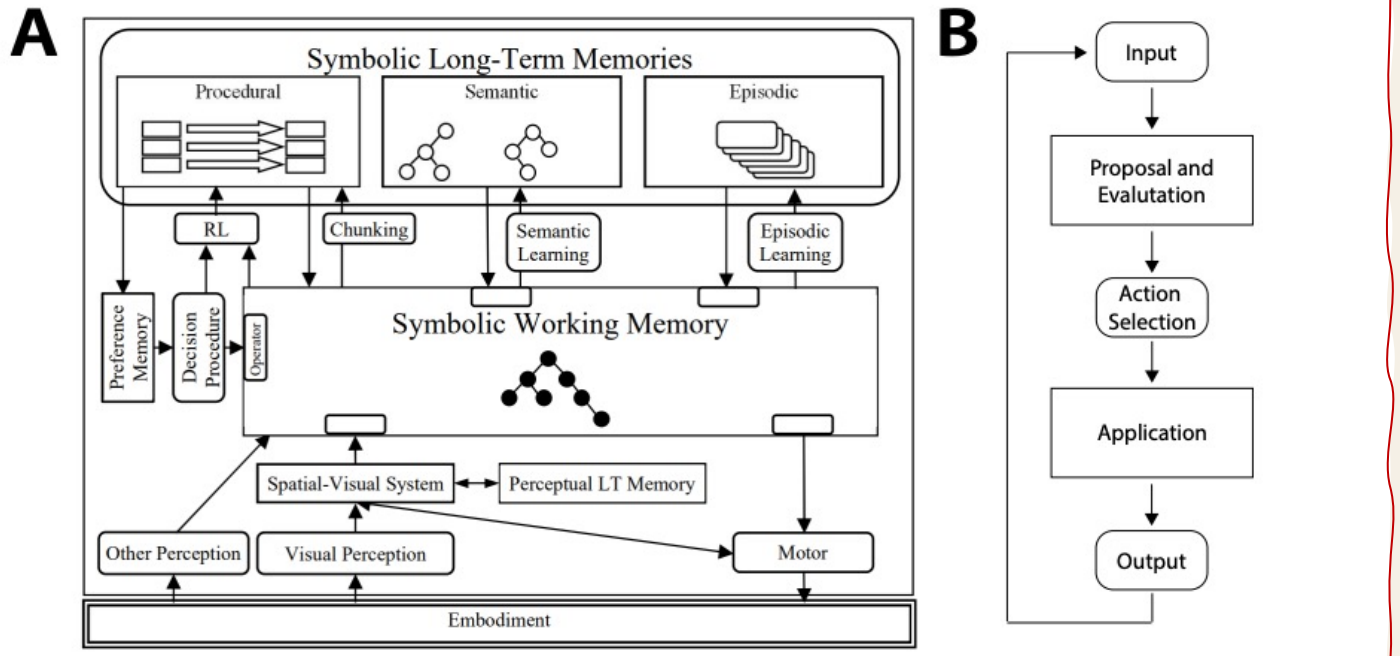


Figure 2: Cognitive architectures augment a production system with sensory groundings, long-term memory, and a decision procedure for selecting actions. **A**: The Soar architecture, reproduced with permission from Laird (2022). **B**: Soar's decision procedure uses productions to select and implement actions. These actions may be *internal* (such as modifying the agent's memory) or *external* (such as a motor command).

A: Retrieval brings knowledge into working memory; learning writes information back.

B: The agent proposes actions, evaluates them, selects one, then executes it.

Architecture and decision cycle: Sumers et al., CoALA, Figure 4.

Events, reflections, and preferences

After Saturday's outing, the assistant asks: *"How did it go?"*
User: *"I visited Garden A and Cafe C. I liked the quiet garden, but the cafe was too crowded."*

Episodic / semantic memory: CoALA §4.1. Reflection: Generative Agents §4.2.

Events, reflections, and preferences

After Saturday's outing, the assistant asks: *"How did it go?"*
User: *"I visited Garden A and Cafe C. I liked the quiet garden, but the cafe was too crowded."*

Episodic memory Records a specific past experience: what happened and its outcome.	Saturday: visited Garden A and Cafe C. Quiet garden liked; cafe crowded. Source: the user's reply above.

Episodic / semantic memory: CoALA §4.1. Reflection: Generative Agents §4.2.

Events, reflections, and preferences

After Saturday's outing, the assistant asks: *"How did it go?"*
User: *"I visited Garden A and Cafe C. I liked the quiet garden, but the cafe was too crowded."*

Episodic memory Records a specific past experience: what happened and its outcome.	Saturday: visited Garden A and Cafe C. Quiet garden liked; cafe crowded. Source: the user's reply above.
Reflection An agent-generated insight inferred from observations or other memories.	Inference: *"May prefer quieter venues."* Tentative: ask before saving it as a lasting preference.

Episodic / semantic memory: CoALA §4.1. Reflection: Generative Agents §4.2.

Events, reflections, and preferences

After Saturday's outing, the assistant asks: *"How did it go?"*
User: *"I visited Garden A and Cafe C. I liked the quiet garden, but the cafe was too crowded."*

Episodic memory Records a specific past experience: what happened and its outcome.	Saturday: visited Garden A and Cafe C. Quiet garden liked; cafe crowded. Source: the user's reply above.
Reflection An agent-generated insight inferred from observations or other memories.	Inference: *"May prefer quieter venues."* Tentative: ask before saving it as a lasting preference.
Stated preference (user fact) A preference the user explicitly states; kept as a fact in semantic memory.	Earlier in the conversation, user said: *"I usually take public transit."* Source: that earlier user statement.

A task run can be an episode. A checkpoint saves execution state for recovery.

Episodic / semantic memory: CoALA §4.1. Reflection: Generative Agents §4.2.

Learning from a human correction

The assistant asks whether the crowded cafe means the user avoids all indoor venues.

Human clarification

“Quiet indoor places are fine. Remember quieter places for future weekend plans.”

Reject this inference

“Dislikes indoor venues”
Crowding did not establish that rule.

Save the confirmed preference

“Prefers quieter venues”
Scope: future weekend plans.

Quiet museums remain eligible. Save the user’s correction and the plans it applies to.

A saved preference needs a source

After the outing, the user says:

“Remember quieter places for future weekends.”

Saved memory (Python dictionary)

```
memory = {
    "content": "Prefers quieter venues",
    "source": "User clarification",
    "statement": ("Remember quieter places "
                  "for future weekends.")
}
```

What to remember

Confirmed by the user.

Where it came from

Keep the speaker and their statement.

Provenance means where a memory came from.

Scope and lifetime of a preference

Confirmed preference: *“Prefers quieter venues.”*

Scope states which plans should use it. Validity states when it stops applying.

“Remember this for future weekends.”



Saved now — Next weekend — User changes it

Status: user-confirmed

Use in future weekend plans.

Update it or stop using it.

“Use this only for today’s outing.”



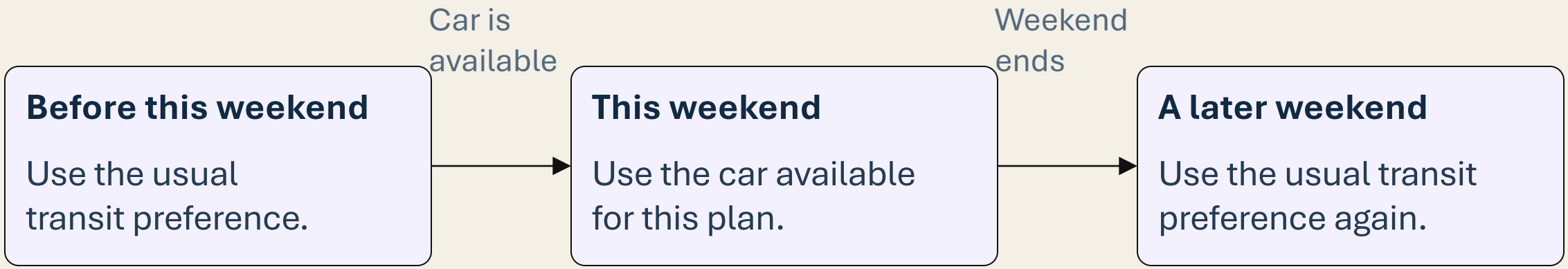
Expires after the outing.

Save the intended scope and when the preference should stop applying.

Temporary Preference

User: *"I have a car this weekend."*

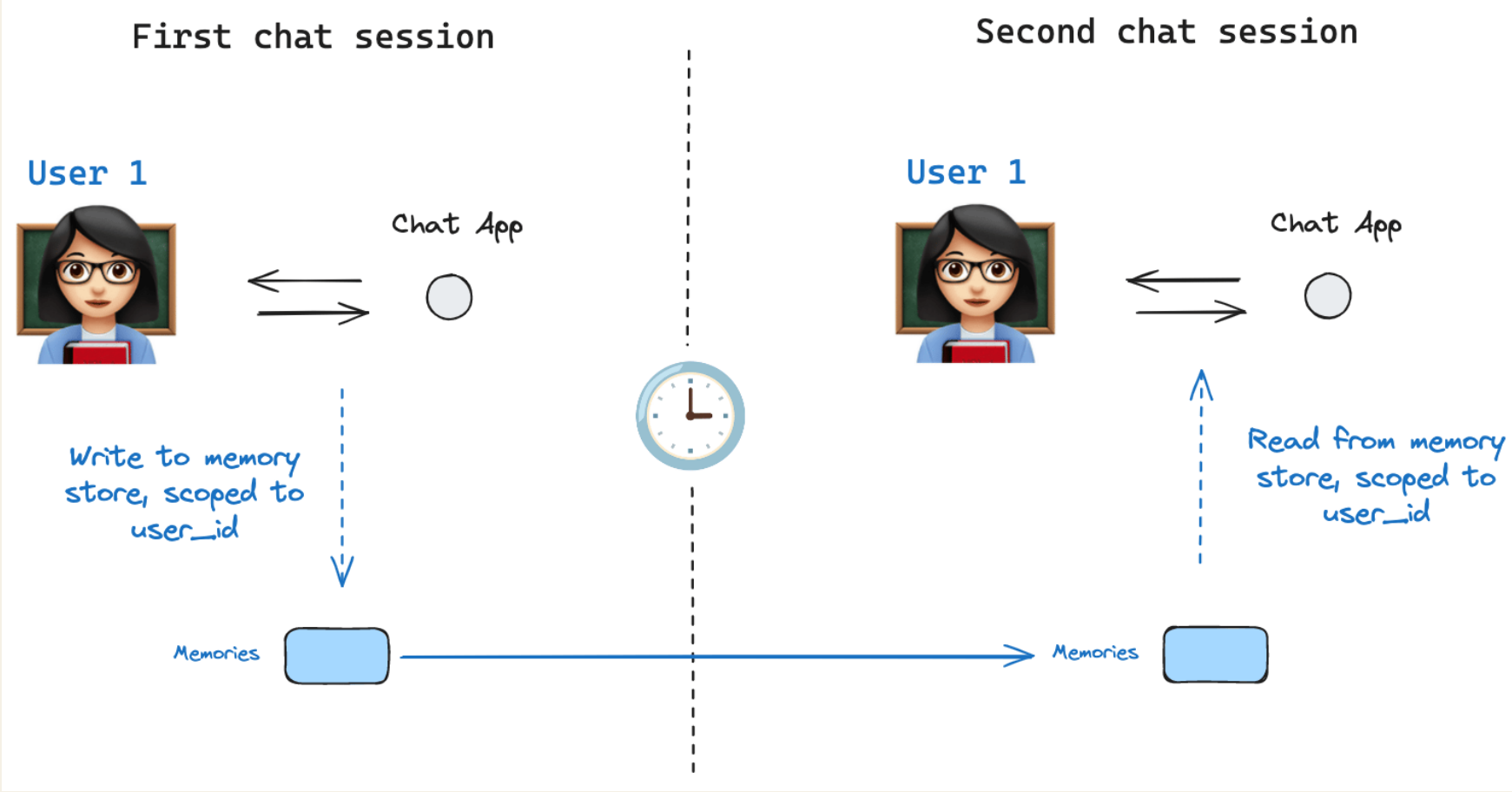
Use the car for this plan. Keep the usual transit preference for later weekends.



Saved in long-term memory: The user usually prefers public transit.
The preference remains saved between planning sessions.
The application includes it in a model call's context when it is relevant.

Temporary preferences: OpenAI tutorial, Step 8.

LangGraph stores share memory across threads



`InMemoryStore` uses RAM. A database-backed store can retain data after exit.

LangGraph Stores: namespace, put, and get (adapted preference example).

LangGraph stores share memory across threads

Save the usual transit preference under the user's namespace.
A later weekend thread can read the same record.

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore()
namespace = ("user-7", "preferences")
store.put(namespace, "transport",
          {"mode": "public transit"})
memory = store.get(namespace, "transport")
print(memory.value)
```

One user namespace

Use user-7's preferences in either thread.

Save a confirmed fact

Save public transit under the transport key.

Read the saved fact

Read the travel mode for this user.

`InMemoryStore` uses RAM. A database-backed store can retain data after exit.

LangGraph Stores: namespace, put, and get (adapted preference example).

Retrieving memory for a new request

The memory is stored. How does a new request find what it needs?

Choose a search method, check whether each result still applies, then include useful evidence in the model call.

The challenge of memory recall

Different queries need different search approaches:

- *“Alice works at Google”*: match the **exact names**.
- *“Where does Alice work?”*: recognize the **meaning** of the question.
- *“What did Alice do last spring?”*: identify the relevant time **period**.
- *“Why did Alice leave?”*: **follow evidence** of a causal relationship.

Four query examples and search needs: Hindsight, “The Challenge of Memory Recall.”

Keys, filters, and keyword search

Start with the information you already know about the requested memory.

- **Exact key lookup:** fetch a record by its user namespace and key.
  Example: get this user's saved transport preference.
- **Metadata or tag filters:** restrict records by user, topic, or status.
  Example: only this user's active outing memories.
- **BM25 keyword search:** rank text using matching terms, their rarity, term frequency, and record length. *"Museum B"* helps find that venue.

A filter controls eligibility. A search score orders eligible candidates.

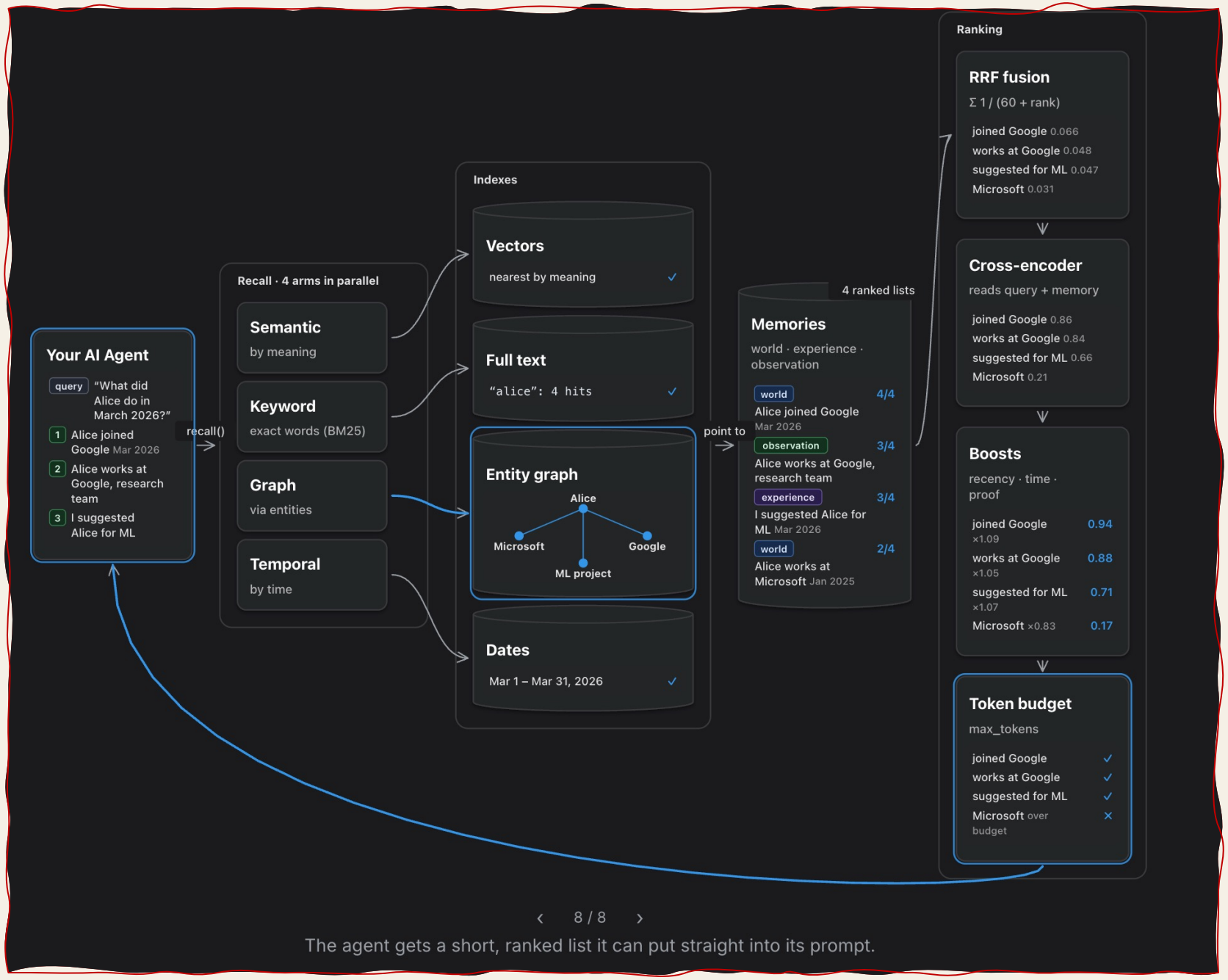
Keys / filters: LangGraph Stores. BM25: Robertson & Zaragoza (2009), §3.4.

Vector search, hybrid retrieval, and reranking

- **Vector search:** embed a query and compare it with stored memory vectors. *“Uncrowded places”* can match *“prefers quieter venues.”*
- **Hybrid retrieval:** combine keyword and vector results to cover names and paraphrases; merge duplicate hits.
- **Query rewriting:** turn a conversational request into an explicit search query, using stated dates and constraints without inventing user preferences.
- **Reranking:** score the query and each candidate together after initial retrieval. This costs extra work, so apply it to a smaller candidate set.

Similarity is a candidate signal. Dates, evidence, and the task determine usefulness.

Hybrid / reranking: Hindsight Recall. Rewriting: Ma et al. (2023).

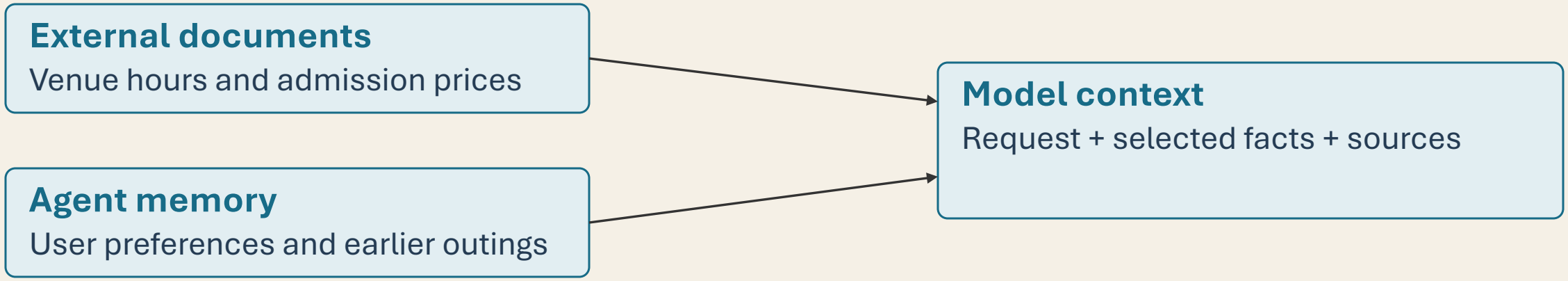


‹ 8 / 8 ›

The agent gets a short, ranked list it can put straight into its prompt.

Storing memories and search indexes

Memory and document search can share a database. Both supply model context.



- **Vector index:** useful for paraphrases across many memories; keys and filters may be enough for a small structured user profile.
- **One database can do both:** PostgreSQL with pgvector stores ordinary fields and embeddings together. A separate vector service is a deployment choice.

RAG pattern: Lewis et al. (2020). Vector storage / search: pgvector README.

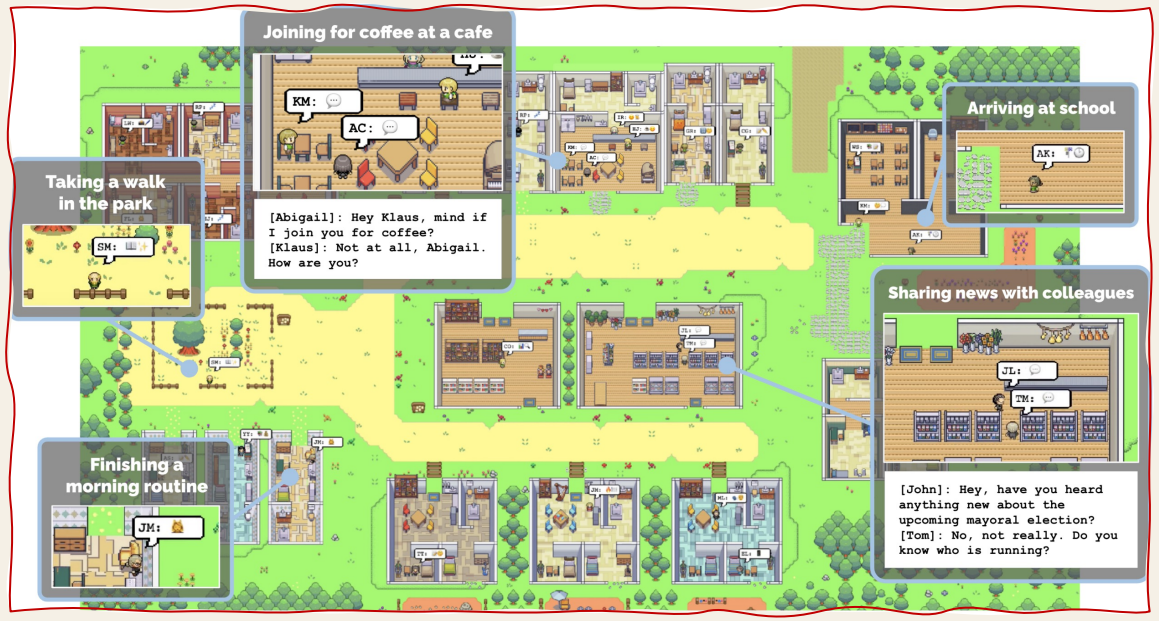
Memory systems in practice

How do systems store memories, retrieve evidence, and revise what they know?

Generative Agents: experiences and reflection.
MemGPT: context and external storage.
Mem0: extracting and updating facts.
Hindsight: linked memories and multiple search methods.

Generative Agents: behavior over time

Park et al. (2023): 25 simulated residents live in Smallville, an interactive town.



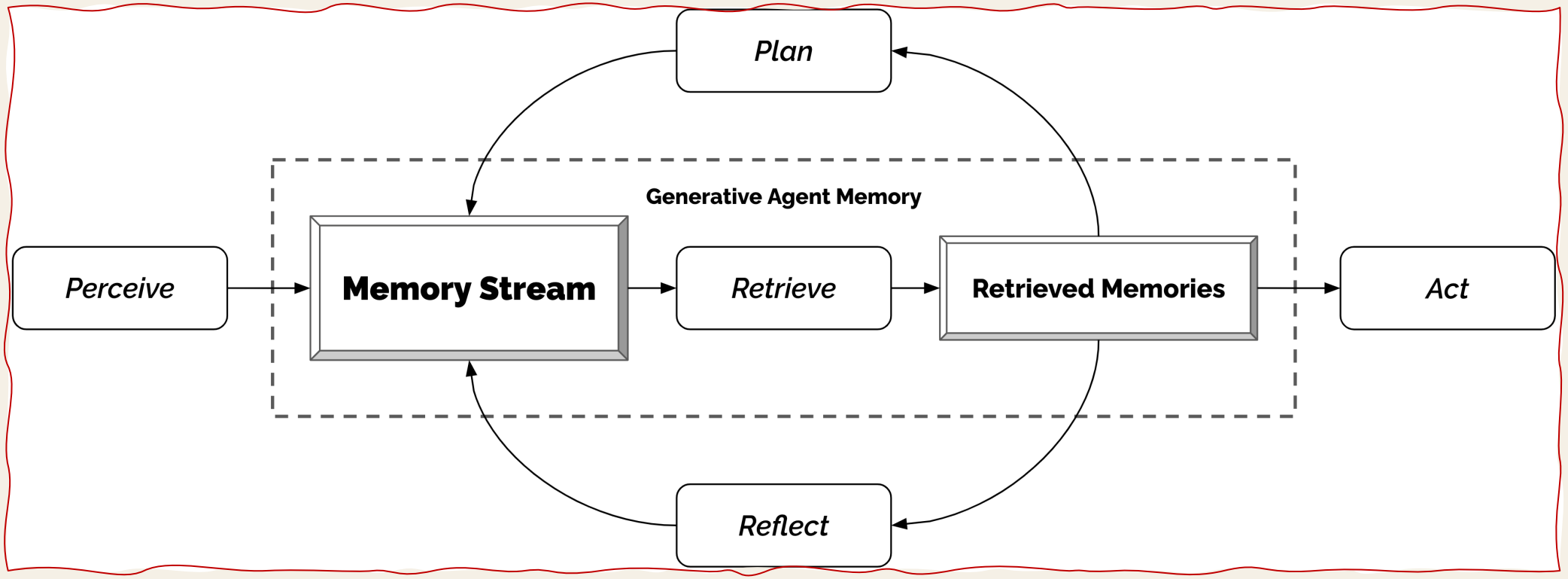
Original Figure 1: Smallville, the simulated town.

- **Problem:** agents must act consistently with earlier experiences.
- **Contribution:** record experiences, retrieve relevant memories, and store reflections to guide behavior.

Stored experiences and reflections guide later behavior.

Smallville setting and Figure 1: Park et al. (2023), Generative Agents.

Generative Agents: memory and reflection



Memory Stream stores experiences. **Retrieve** selects memories for the model. **Reflect** writes broader conclusions back into memory.

Original Figure 5: memory architecture. Park et al. (2023).

How Generative Agents ranks memories

Park et al. combine three scores to choose which memories enter the model's context.

Retrieval scores and selection: Park et al., §4.1.

How Generative Agents ranks memories

Park et al. combine three scores to choose which memories enter the model's context.

Relevance	Does it match the current situation? Similarity between the memory and the retrieval query.

Retrieval scores and selection: Park et al., §4.1.

How Generative Agents ranks memories

Park et al. combine three scores to choose which memories enter the model's context.

Relevance	Does it match the current situation? Similarity between the memory and the retrieval query.
Recency	How recently was it retrieved? The score decays with time since the last retrieval.

Retrieval scores and selection: Park et al., §4.1.

How Generative Agents ranks memories

Park et al. combine three scores to choose which memories enter the model's context.

Relevance	Does it match the current situation? Similarity between the memory and the retrieval query.
Recency	How recently was it retrieved? The score decays with time since the last retrieval.
Importance	How significant is the experience? The model scores significance when creating the memory.

Choose the highest-ranked memories that fit the model's context limit.

Retrieval scores and selection: Park et al., §4.1.

Memory Retrieval

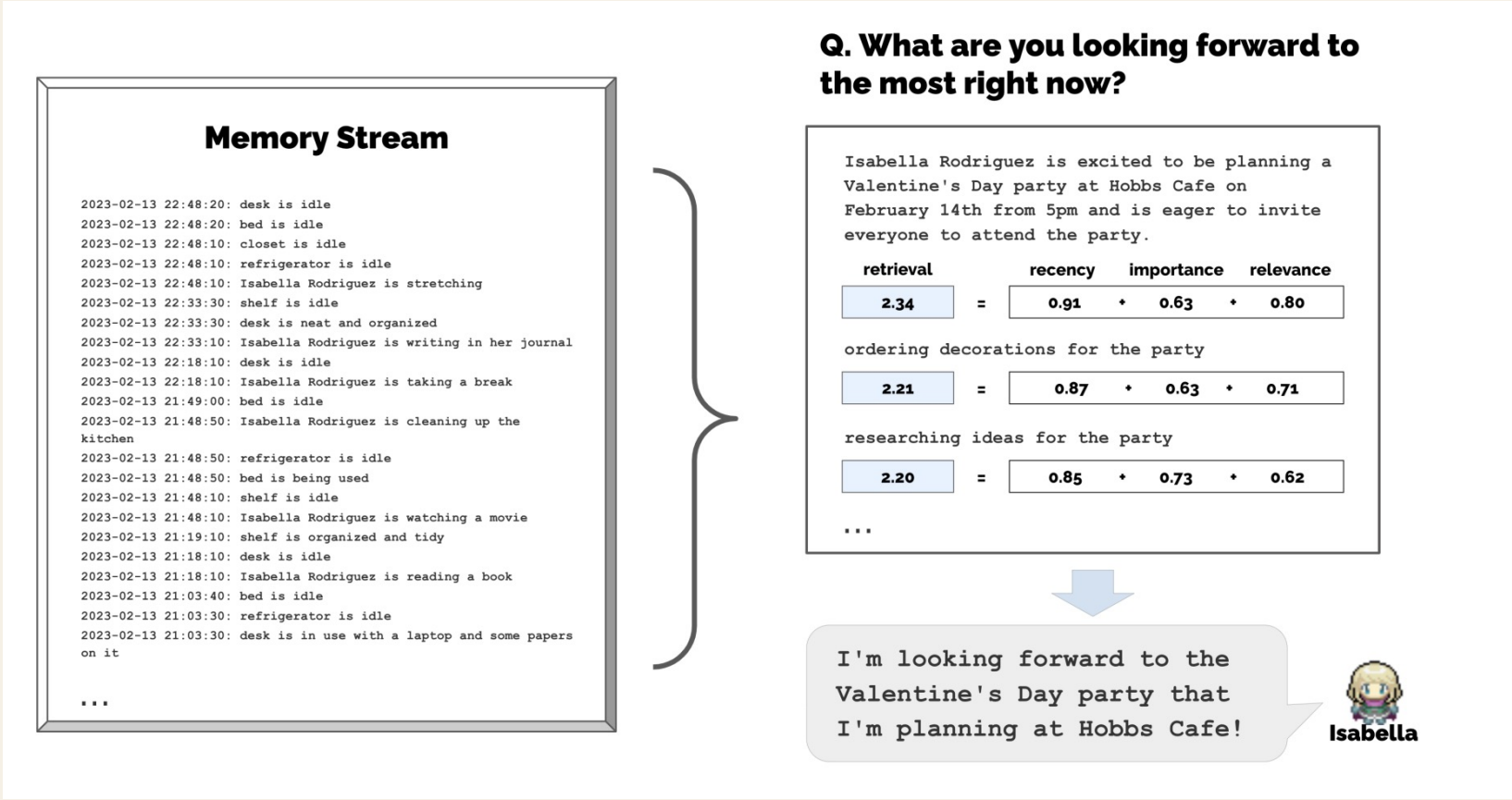


Figure 6. The memory stream comprises a large number of observations that are relevant and irrelevant to the agent's current situation. Retrieval identifies a **subset of these observations** that should be passed to the language model to condition its response to the situation.

GENERATIVE AGENTS // MEMORY CONSOLIDATION

How Agents Reflect: From Daily Events to Core Beliefs

01

Know When to Pause

Triggers after importance scores passes 150.

02

Review Recent Life

Pulls the 100 latest memory stream entries.

03

Ask 3 Big Questions

Finds 3 broad questions worth exploring.

04

Gather Past Evidence

Searches all history for matching memories.

05

Infer 5 New Insights

Saves 5 cited reflections back to memory.

STEP 03 PROMPT // GENERATING 3 QUESTIONS

"Given only the information above, what are 3 most salient high-level questions we can answer about the subjects in the statements?"

STEP 05 PROMPT // EXTRACTING 5 INSIGHTS

"What 5 high-level insights can you infer from the above statements? (example format: insight (because of 1, 5, 3))"

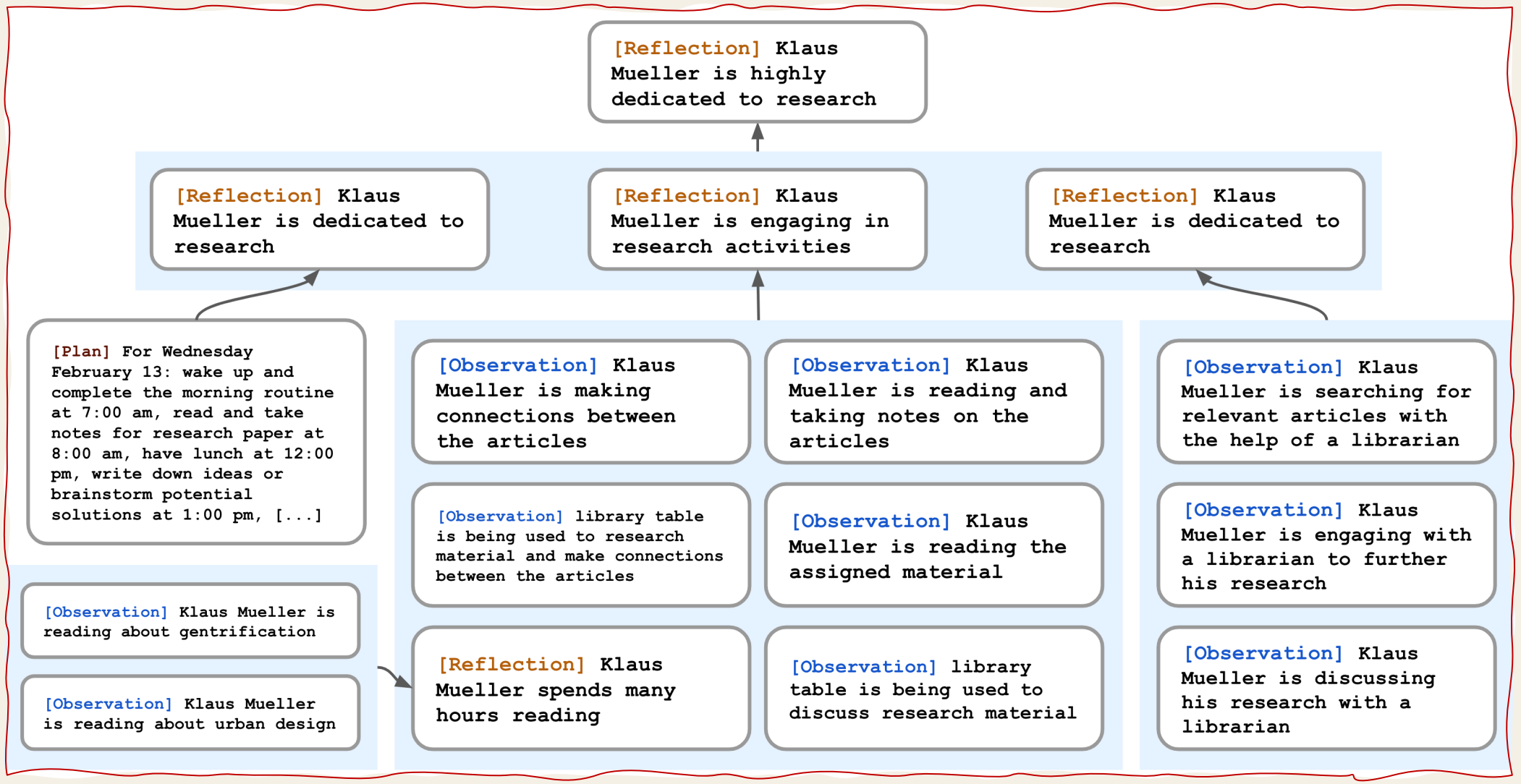


Figure 7: A reflection tree for Klaus Mueller. The agent's observations of the world, represented in the leaf nodes, are recursively synthesized to derive Klaus's self-notion that he is highly dedicated to his research.

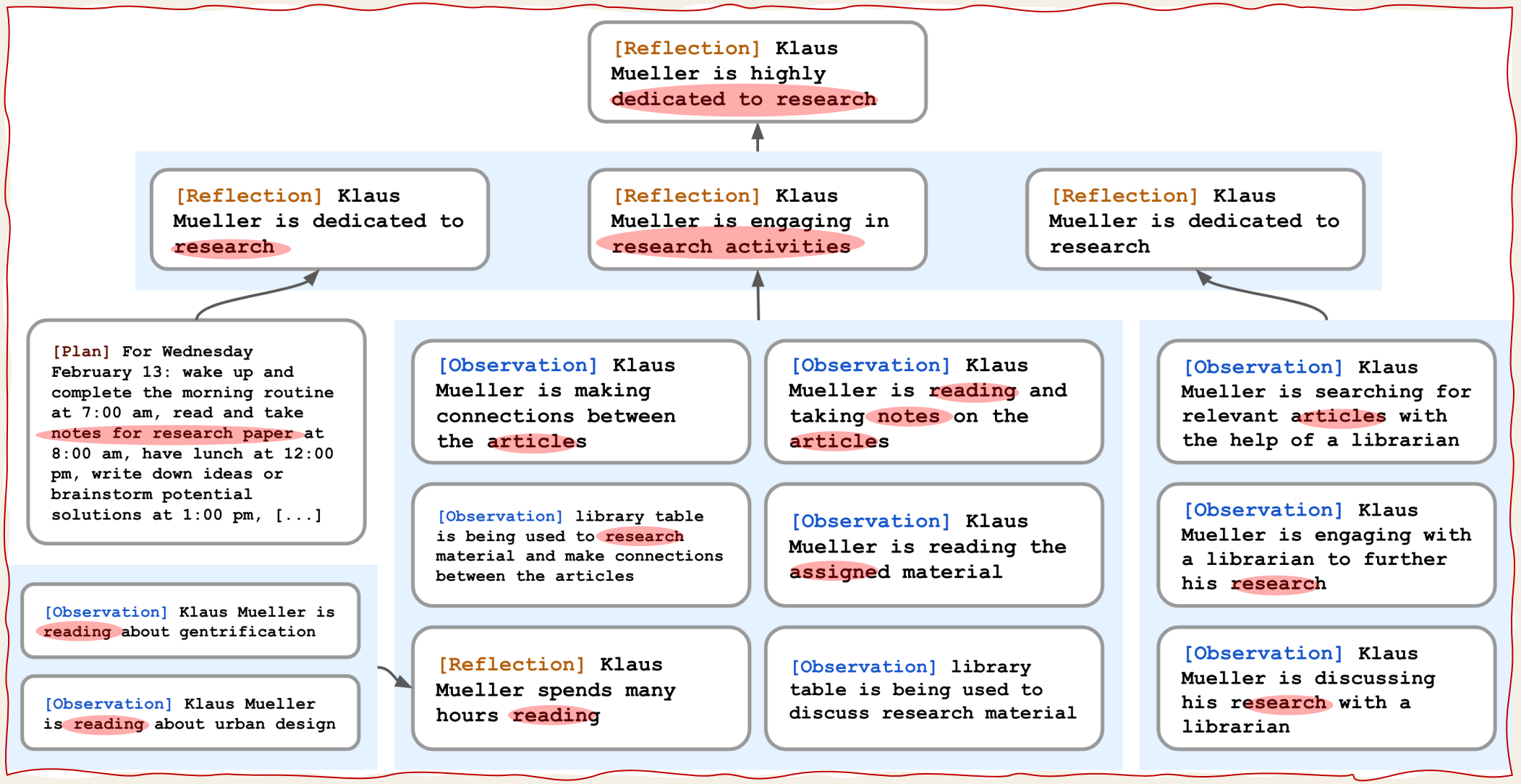


Figure 7.A reflection tree for Klaus Mueller. The agent's observations of the world, represented in the leaf nodes, are recursively synthesized to derive Klaus's self-notion that he is highly dedicated to his research.

Architecture Ablation Study

- **Full agent (Red bar):** Agents with observation memory, reflection, and planning scored the highest
  - Even beating human crowdworkers asked to role-play the same characters.
- **Removing pieces one by one (Grey bars):** Turning off planning, then reflection, and finally memory retrieval caused believability to drop at every step.
- **Base LLM alone:** Without access to stored memories, reflections, or plans, agents forgot relationships, made up past events, and repeated daily

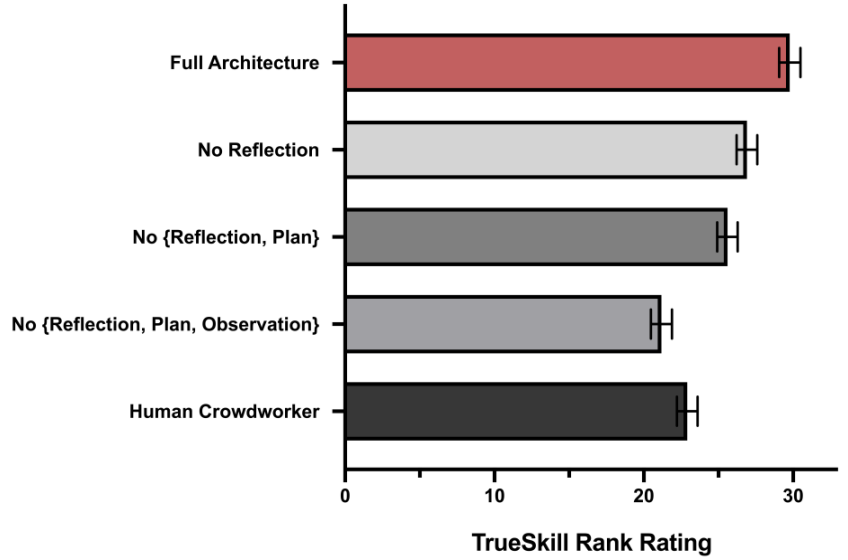


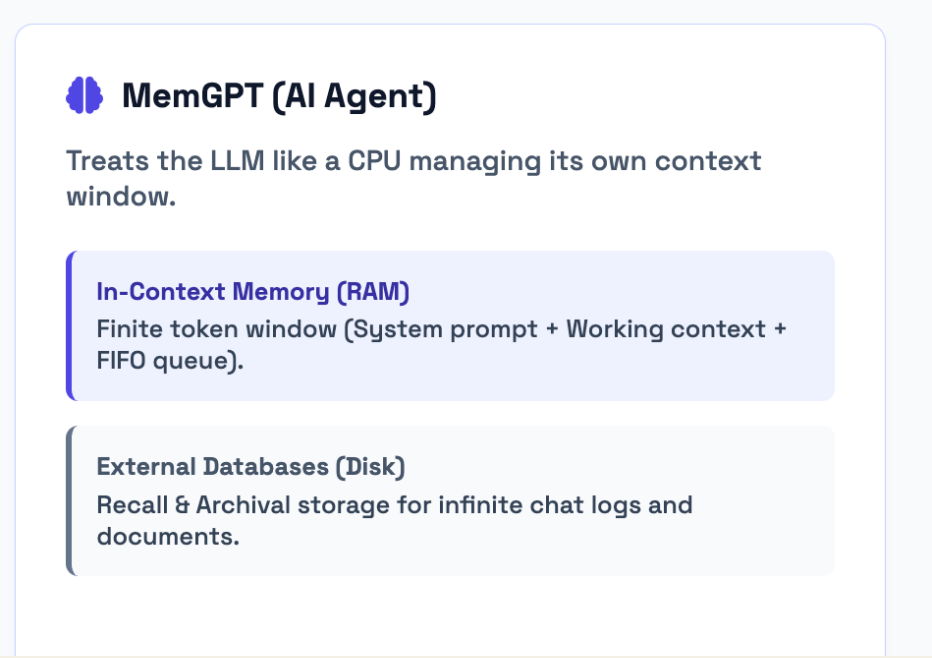
Figure 8: The full generative agent architecture produces more believable behavior than the ablated architectures and the human crowdworkers. Each additional ablation reduces the performance of the architecture.

The authors measured **believability** by running a two-day simulation of Smallville, "freezing" the agents to interview them with 25 probing questions, and having 100 human evaluators rank the answers from different agent versions side by side.

MemGPT: managing a finite context

A memory-management system for long conversations and document analysis.

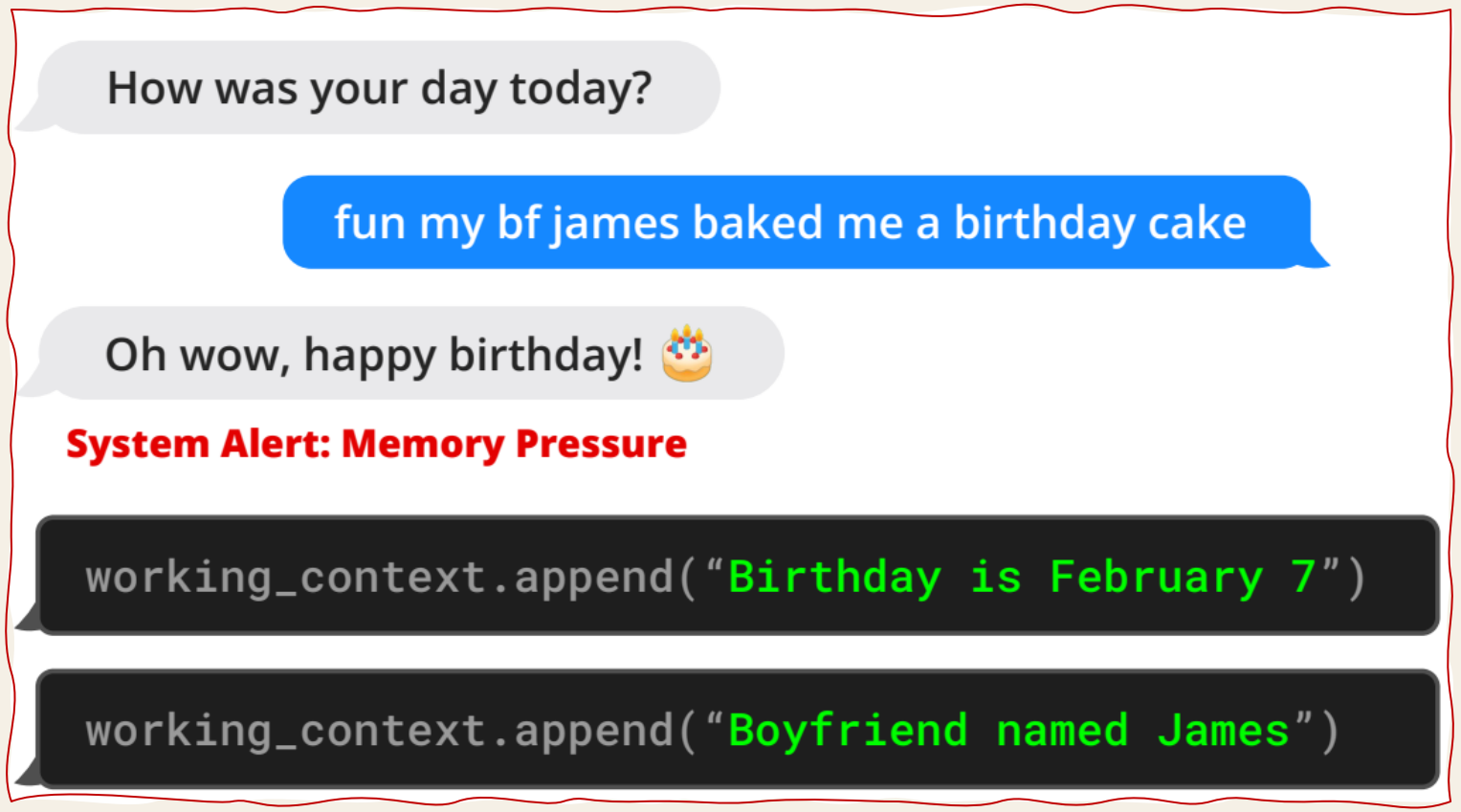
- **Problem:** one model call cannot include every earlier interaction.
- **Contribution:**
  - LLM calls tools to manage its own context
  - Applying Virtual Memory in Operating Systems to agents



Retained information becomes useful when the agent brings it into context.

Finite-context problem and memory hierarchy: Packer et al., Abstract and §2.

MemGPT: writing key facts before context fills

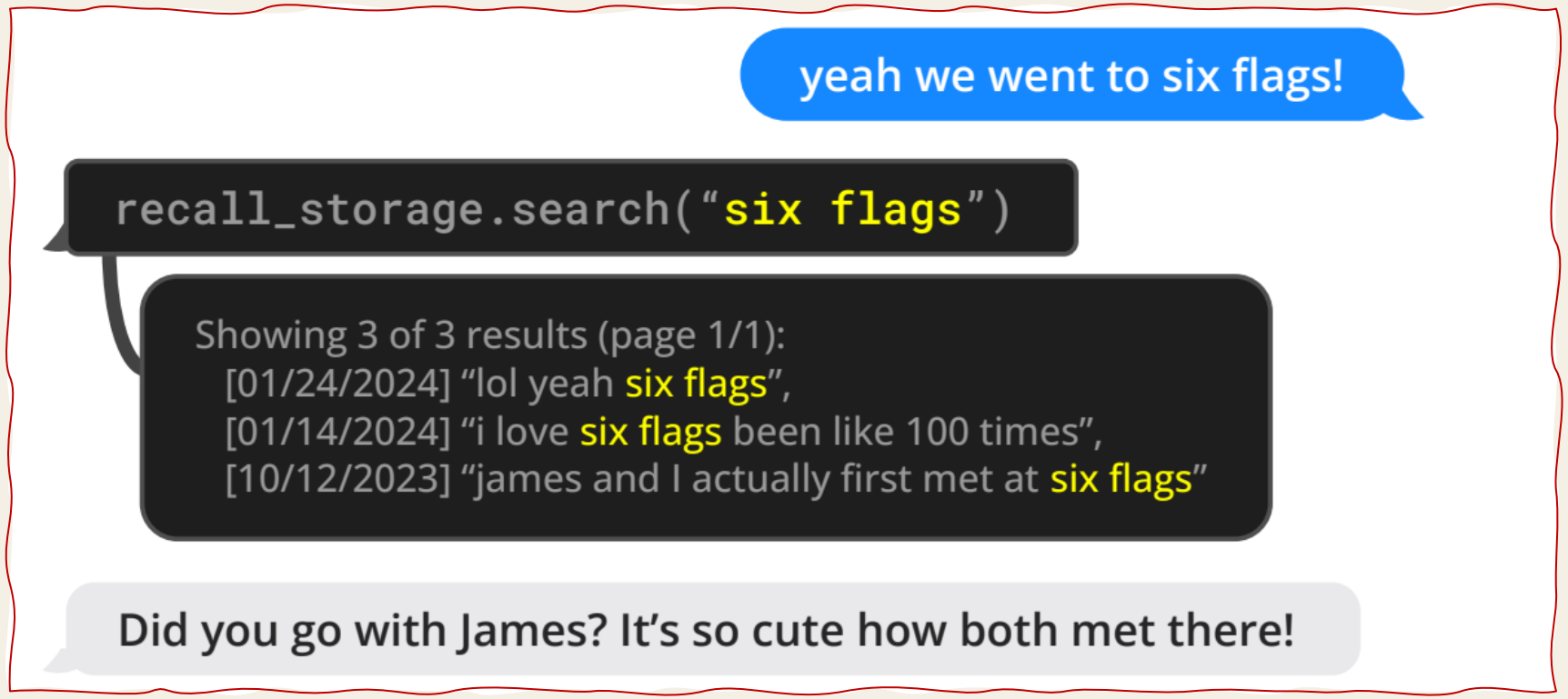


Original Figure 1
Writing key facts

- **Trigger:** context is nearly full.
- **Write:** add facts to Working Context.
- **Retain:** save full messages in recall storage.

Original Figure 1: writing facts under context pressure. Packer et al.

MemGPT: retrieving an earlier conversation



Original Figure 2
Retrieving history

- **Request:** the user mentions Six Flags.
- **Retrieve:** find where the couple first met.
- **Use:** add that detail to the reply.

Original Figure 2: retrieving earlier messages. Packer et al.

MemGPT: updating context

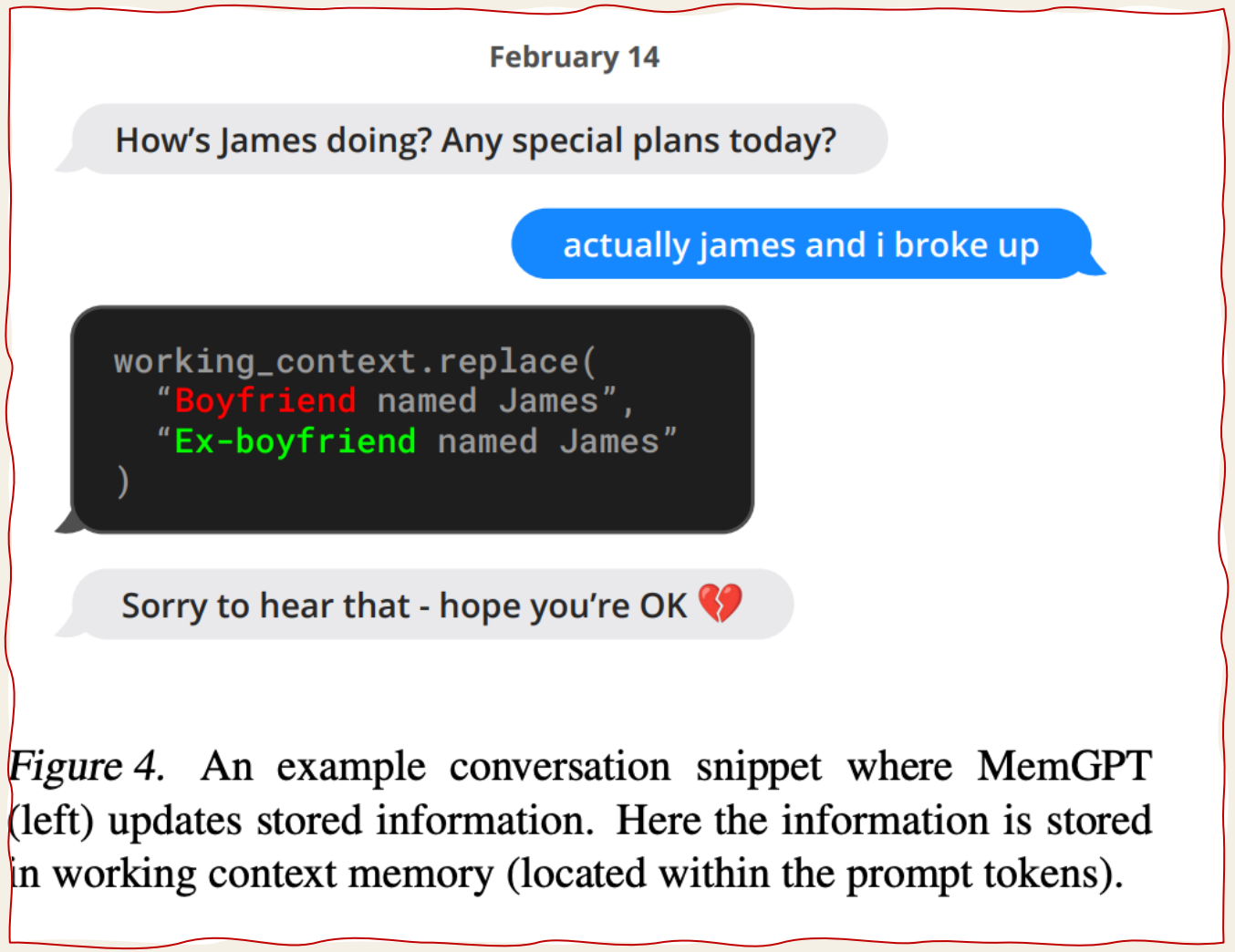
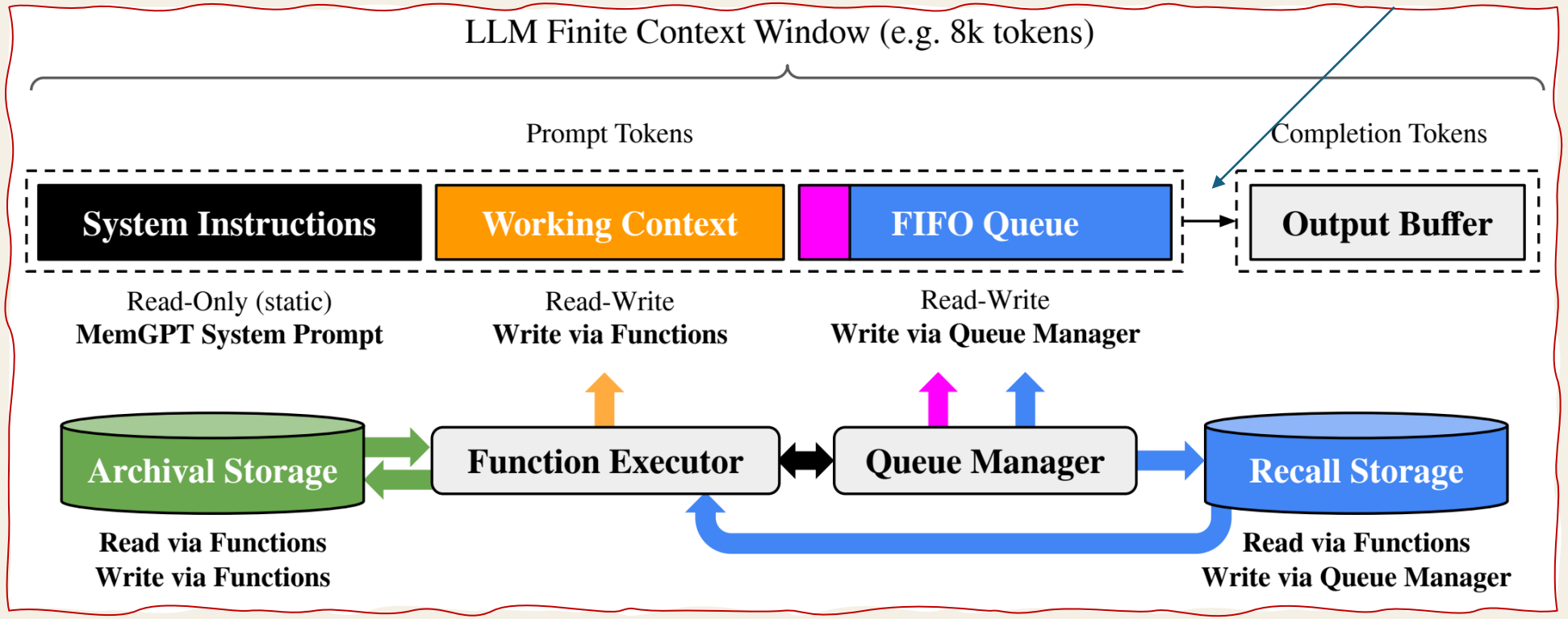


Figure 4. An example conversation snippet where MemGPT (left) updates stored information. Here the information is stored in working context memory (located within the prompt tokens).

MemGPT's context and storage hierarchy

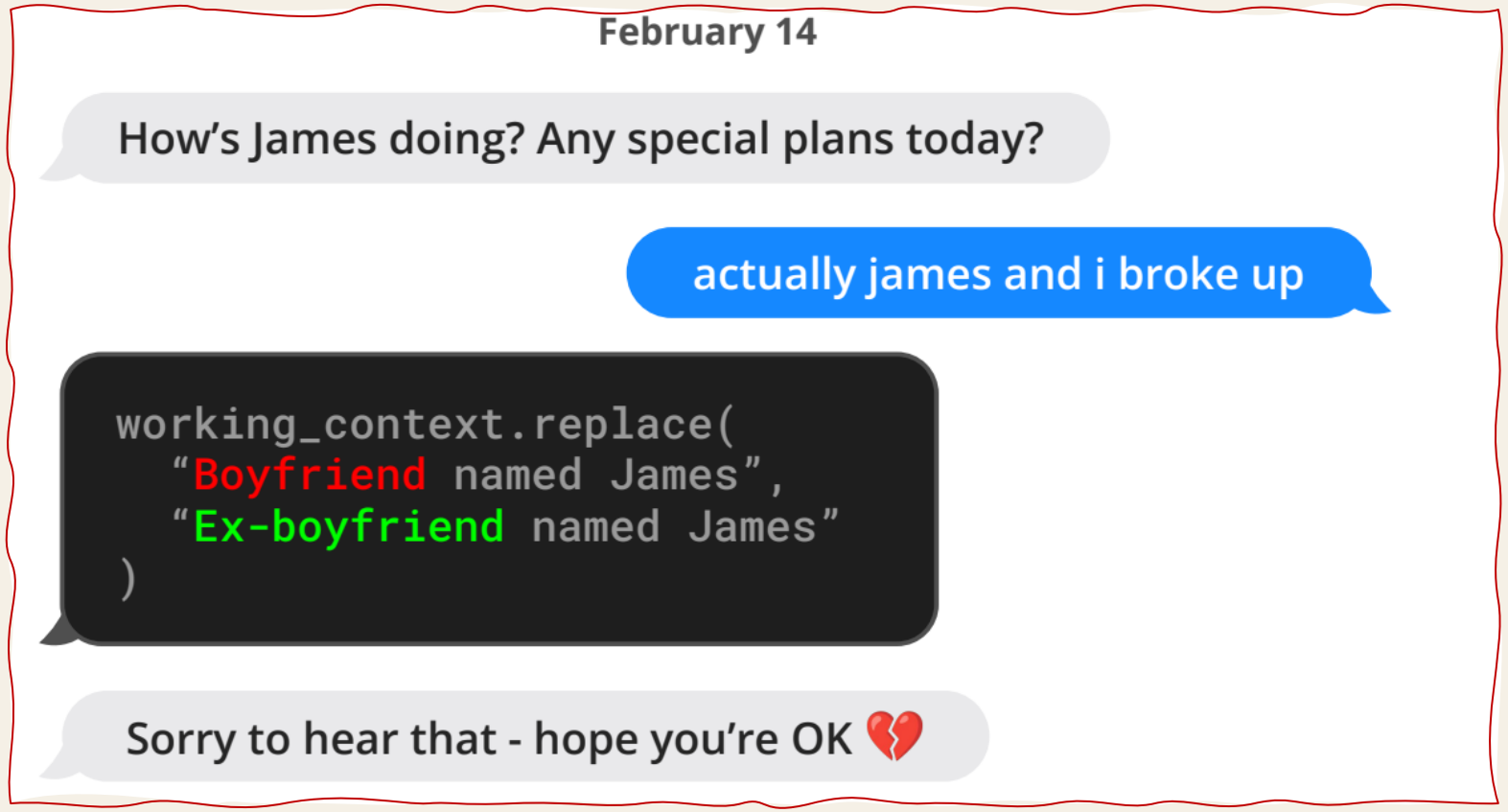
LLM sits here



The Queue Manager automatically handles the rolling chat window (saving overflow to Recall Storage and keeping a pink summary at the front)

Function Executor lets the LLM actively edit its orange Working Context scratchpad or search/write to Archival and Recall Storage whenever it needs information that isn't currently in the prompt.

MemGPT: correcting a stored relationship



Original Figure 4
Correcting a fact

- **Update:** the relationship has ended.
- **Correct:** replace the fact in Working Context.
- **Limit:** old copies can remain in history.

Original Figure 4: correcting Working Context. Packer et al.

MemGPT performance

An LLM judges the answers.

Table 2. **Deep memory retrieval (DMR) performance.** In this task, the agent is asked a specific question about a topic discussed in a prior conversation (sessions 1–5). The agent's response is scored against the gold answer. MemGPT significantly outperforms the fixed-context baselines.

Model	Accuracy ⇑	ROUGE-L (R) ⇑
GPT-3.5 Turbo	38.7%	0.394
+ MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+ MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
+ **MemGPT**	**93.4%**	**0.827**

Table 3. **Conversation opener performance.** The agent's conversation opener is evaluated using similarity scores to the gold persona labels (SIM-1/3) and to the human-created opener (SIM-H). MemGPT is able to exceed the performance of the human-created conversation opener with a variety of underlying models.

Method	⇑ SIM-1	SIM-3	SIM-H
Human	0.800	0.800	1.000
GPT-3.5 Turbo	0.830	0.812	**0.817**
GPT-4	**0.868**	**0.843**	0.773
GPT-4 Turbo	0.857	0.828	0.767

Mem0: keeping conversational memory current

Chhikara et al. (2025): extract and update facts across conversations.

- **Problem:** a new chat can lose earlier user constraints.
- **Contribution:** extract facts, compare related memories, update storage.

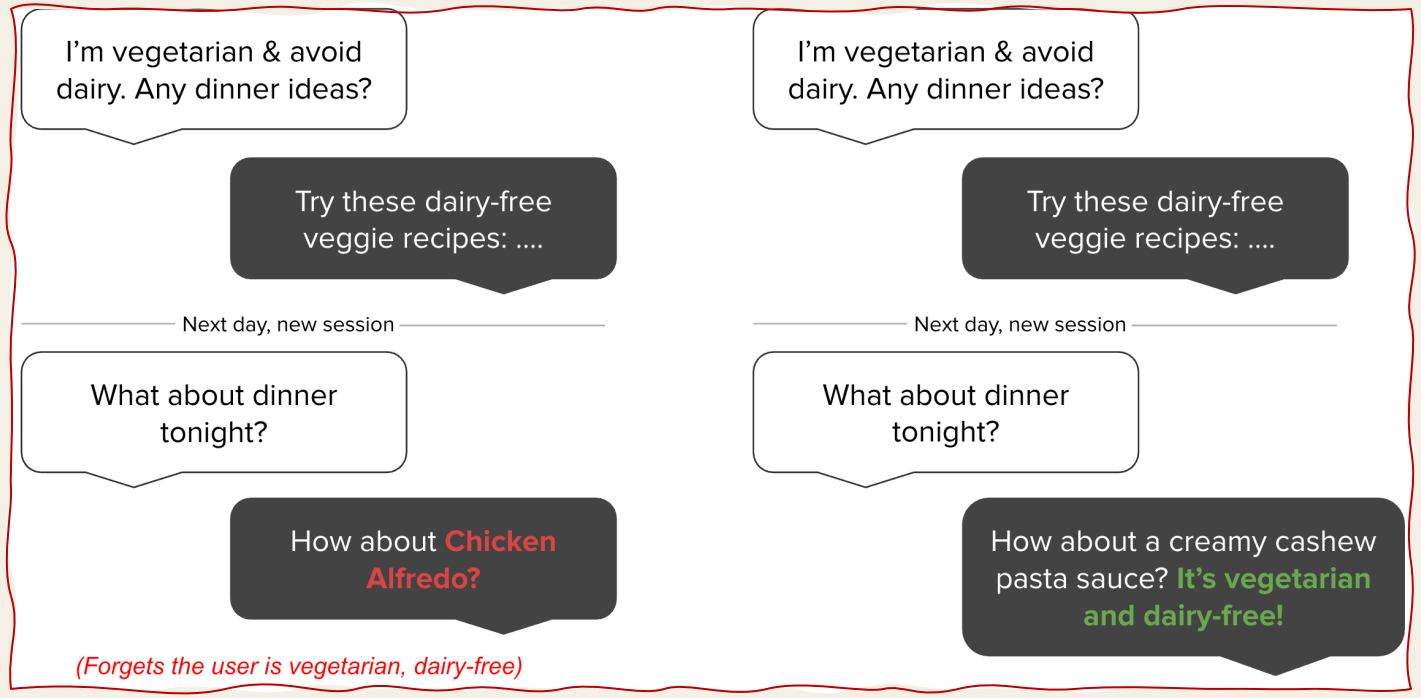


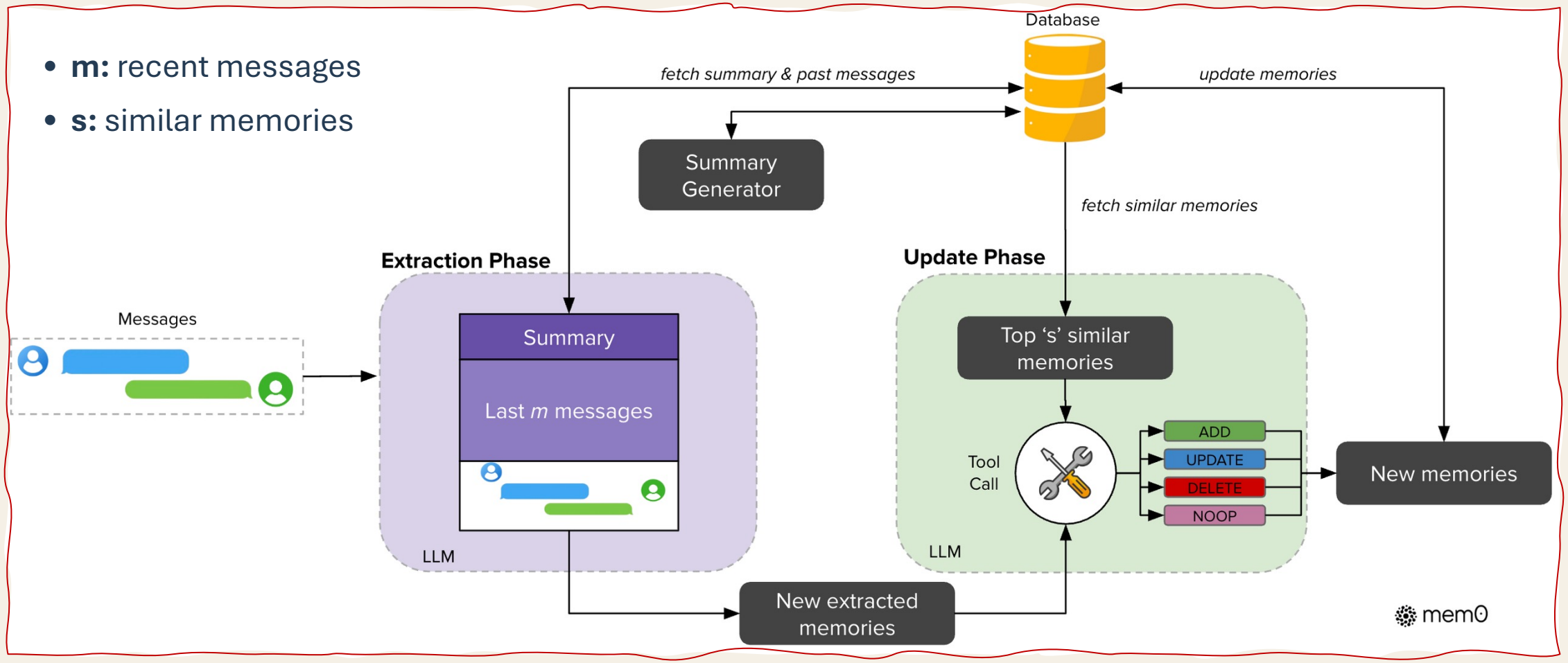
Figure 1: vegetarian and dairy-free constraints should survive a new session.

Extract useful facts; revise related memories as conversations change.

Mem0 problem and contribution: Chhikara et al., Figure 1 and §2.1.

Mem0: extracting and updating memories

- **m:** recent messages
- **s:** similar memories



Original Figure 2: extraction, update, and asynchronous summary refresh.

Mem0’s memory update choices

Mem0 defines four memory operations in §2.1. Here is what each would do in the weekend planner.

Operation	Relationship to stored memory	Weekend planner illustration
`ADD`	Useful new information	Save a newly confirmed preference
`UPDATE`	Complements an existing memory	Add the confirmed weekend scope
`DELETE`	Contradicts an existing memory	Remove the outdated current location
`NOOP`	Adds no useful information	Leave an existing duplicate unchanged

The application decides which proposed writes and deletions it permits.

Four memory operations: Mem0, §2.1.

Mem0: save and retrieve a preference

The repository provides memory APIs, a server, and hosted integration.

Setup: pip install mem0ai; configure model and embedding credentials.

```
from mem0 import Memory
memory = Memory()
messages = [{"role": "user",
             "content": "I prefer quiet places."}]
memory.add(messages, user_id="weekend-user")
hits = memory.search(
    query="Where should I go Saturday?",
    filters={"user_id": "weekend-user"}, top_k=3)
for hit in hits["results"]:
    print(hit["memory"])
```

Save this user's preference.

Find memories for this user.

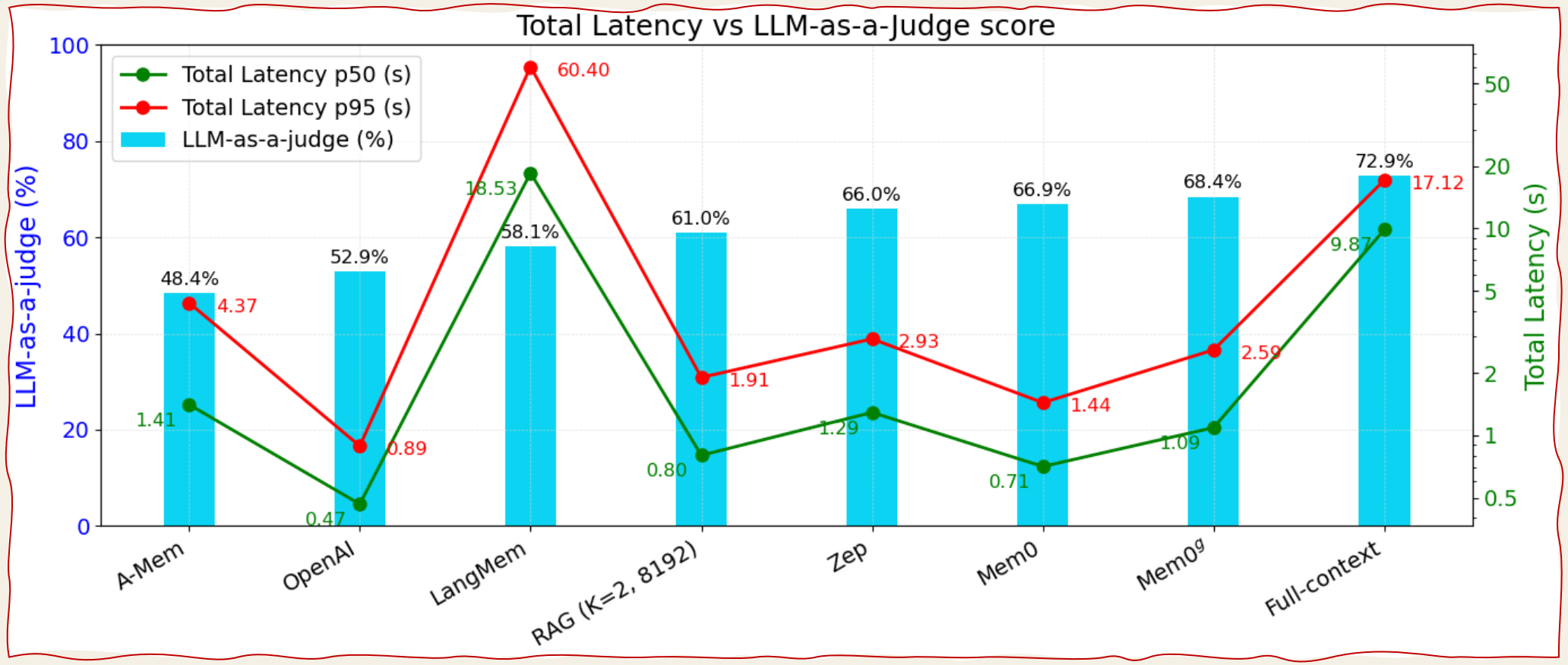
Current extraction is ADD-only. The four-operation diagram describes the 2025 paper.

Mem0 repository: library setup, Basic Usage, and April 2026 algorithm changes.

Mem0's quality and latency tradeoff

Figure 4b: LoCoMo quality (bars) and total response time (lines).

p95: 95% of requests finish within this time. Time axis: logarithmic.



Mem0: 66.9%, 1.44 s p95. Full context: 72.9%, 17.12 s p95.

Maintaining memory over time

What should the assistant keep, combine, update, or remove as conversations accumulate?

Keep the evidence when combining facts.
Do not turn a temporary exception into a lasting preference.

Pruning long-term memory

Pruning keeps useful memories and retires stale or unnecessary records.

- **Keep:** the confirmed quiet-venue preference still helps future plans.
- **Merge:** combine duplicate quiet-place notes, keeping both source references.
- **Expire:** *“I have a car this weekend”* stops applying after that weekend.
- **Archive:** retain an old itinerary for history, outside routine retrieval.
- **Delete:** remove a withdrawn address record and its search entries.

Keep source, date, scope, and uncertainty when merging or archiving.
Excluding a record from one model call leaves it stored. Deletion changes storage.

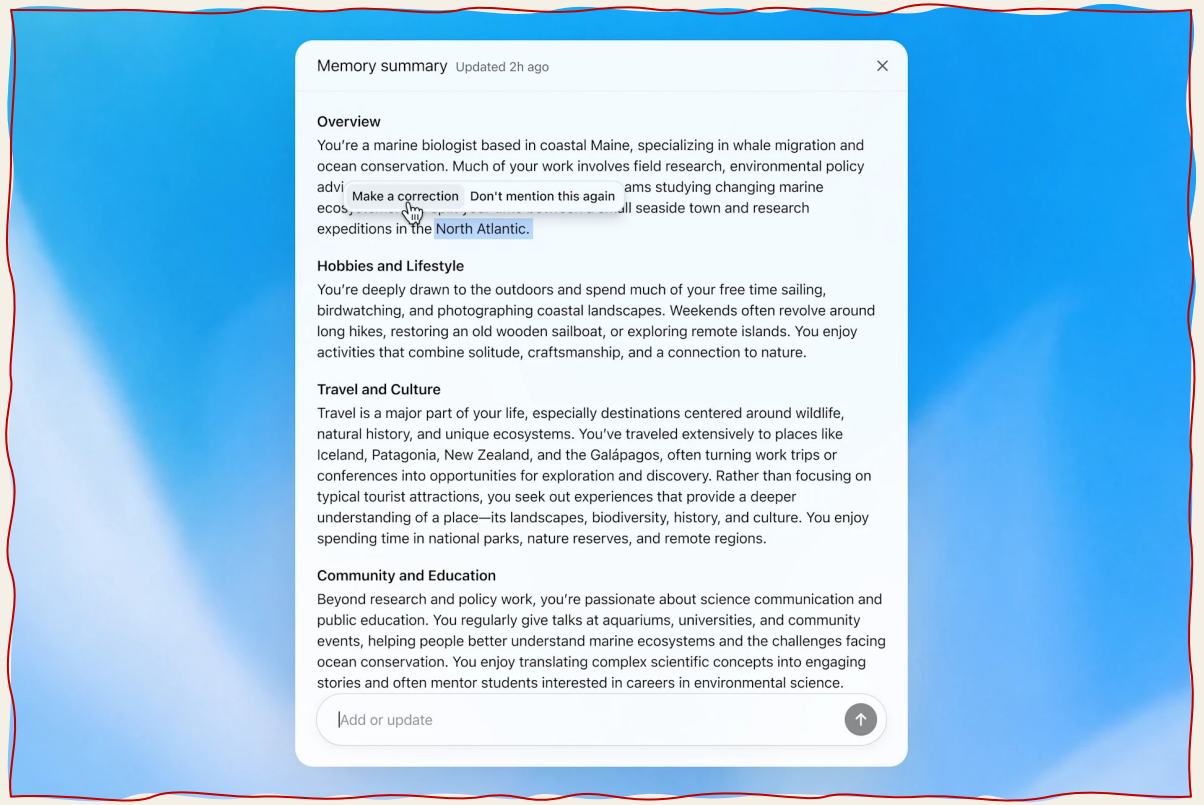
Stored memory and updates: LangGraph Memory overview.

Background memory consolidation

"Dreaming" synthesizes memory across conversations in a background process.

- **Review:** read conversations and existing memories.
- **Reconcile:** merge duplicates; correct contradicted facts.
- **Derive:** infer useful insights from evidence. Keep them tentative until confirmed.

Review the summary; correct errors.
Keep evidence links and dates.
Confirm inferred preferences.



Background synthesis: OpenAI (2026). Reconciliation: Anthropic Dreams.

Anthropic Dreams: separate output memory

An asynchronous job reviews an existing store and past session transcripts.
Setup: authenticated client; existing store_id, session_a, and session_b.

```
dream = client.beta.dreams.create(
    inputs=[
        {"type": "memory_store",
         "memory_store_id": store_id},
        {"type": "sessions",
         "session_ids": [session_a, session_b]},
    ],
    model="claude-opus-5-5",
    instructions="Preserve evidence and uncertainty.",
)
```

Existing store + past sessions

Steer what the job keeps and combines.

Merge duplicates, update stale facts, and derive insights with evidence.
Input stays unchanged. Review the separate output store before using it.

Anthropic Dreams: “How it works,” API example, and “Use the output.”

Testing memory behavior

How do we know the assistant recovered, remembered the right fact, or stopped using an old one?

Check behavior to test useful recall. Inspect storage to verify deletion.

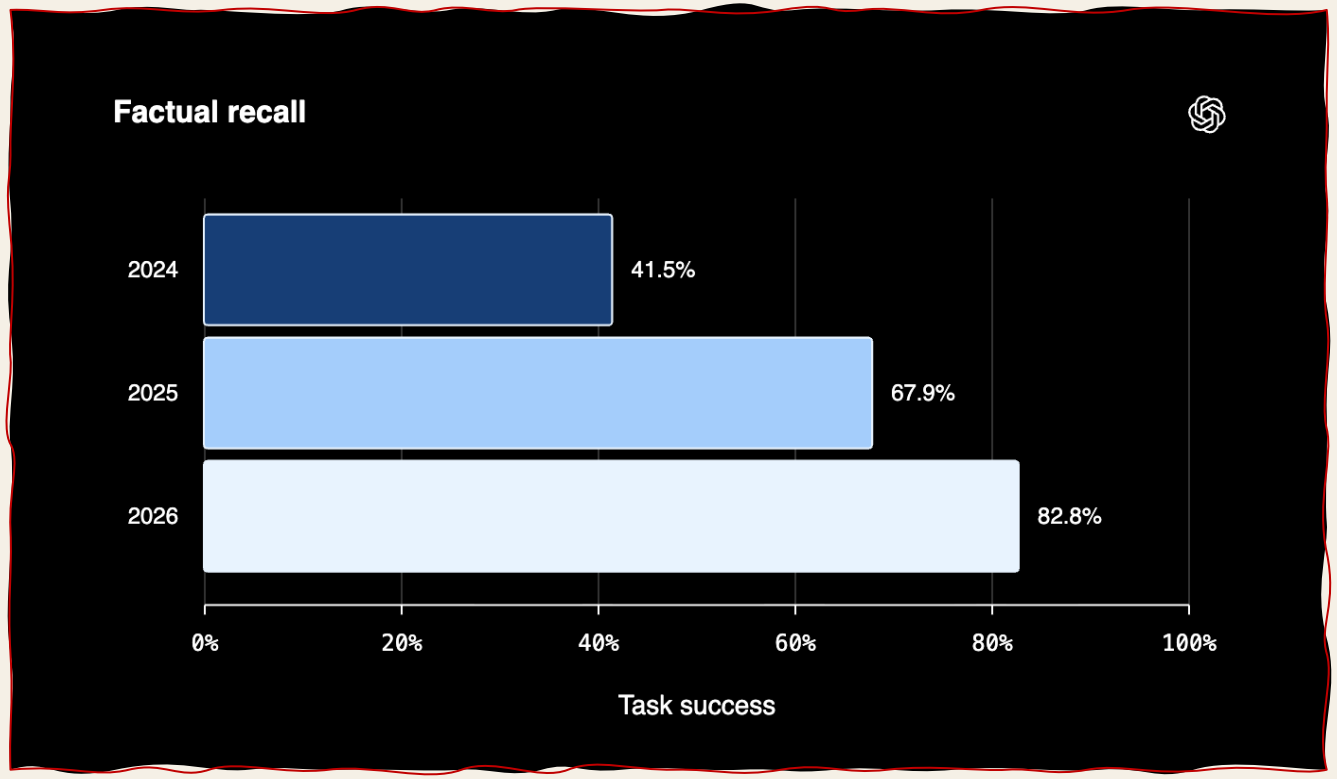
OpenAI: evaluating useful memory

- **What is good memory?**
  - **Carry forward useful context:** You tell ChatGPT something once, and it remembers that information in your subsequent chats.
  - **Follow preferences and constraints:** If you describe a preference (e.g., you're vegetarian), then ChatGPT should take actions that are consistent with that preference going forward.
  - **Stay current over time:** Memory should account for the passage of time. Imagine "The user is planning their birthday party for next Saturday"; eventually, Sunday arrives.

evaluating ChatGPT Plus and Pro memory
2024: saved memories
2025: + dreaming V0
2026: dreaming V3

OpenAI, "How we evaluate memory" and original factual-recall chart (2026).

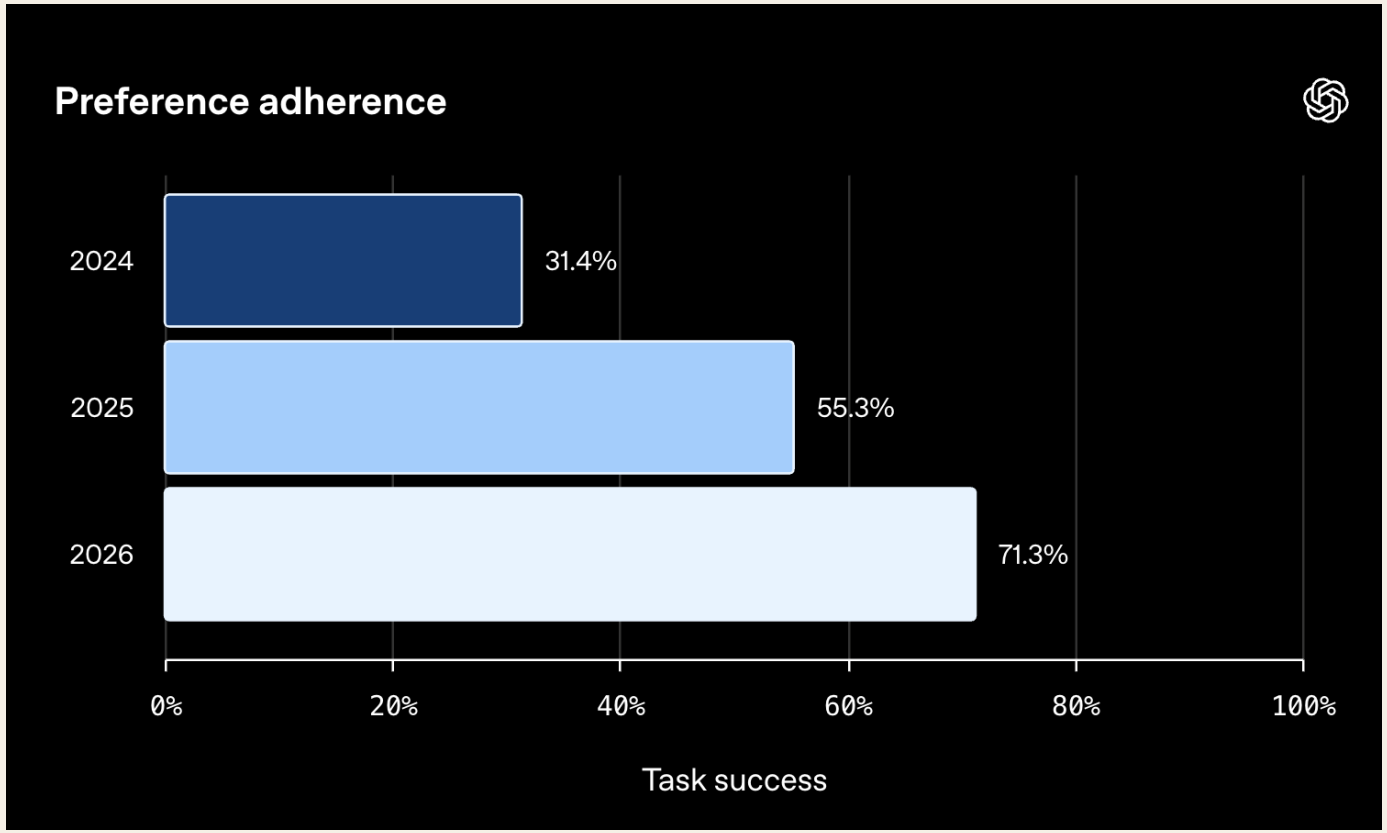
OpenAI: Carrying Forward



evaluating ChatGPT Plus and Pro memory
2024: saved memories
2025: + dreaming V0
2026: dreaming V3

OpenAI, “How we evaluate memory” and original factual-recall chart (2026).

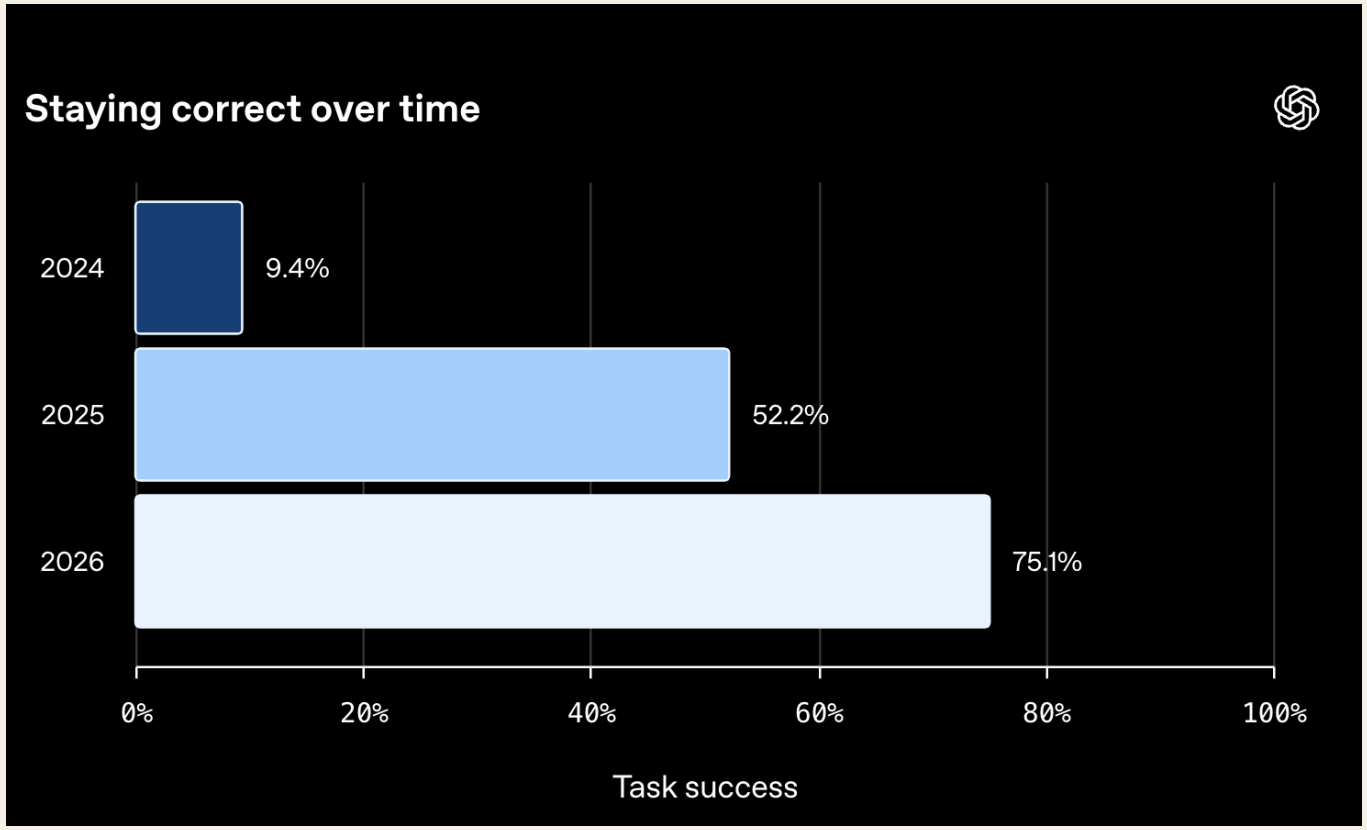
OpenAI: Follow Preferences



evaluating ChatGPT Plus and Pro memory
2024: saved memories
2025: + dreaming V0
2026: dreaming V3

OpenAI, “How we evaluate memory” and original factual-recall chart (2026).

OpenAI: Staying Correct Over Time



evaluating ChatGPT Plus and Pro memory
2024: saved memories
2025: + dreaming V0
2026: dreaming V3

OpenAI, “How we evaluate memory” and original factual-recall chart (2026).

MemoryAgentBench: testing more than recall

Hu, Wang, and McAuley (2025/2026) test how agents use information supplied across successive turns.

- **Problem:** recalling one fact does not show whether an agent can use or correct it.
- **Contribution:** separately test **recall**, **learning** from examples, **combining** a long history, and **replacing** outdated facts.

Evaluate remembering and correcting as separate abilities.

Benchmark purpose and contribution: Hu et al., Abstract and Introduction.

MemoryAgentBench: four memory abilities

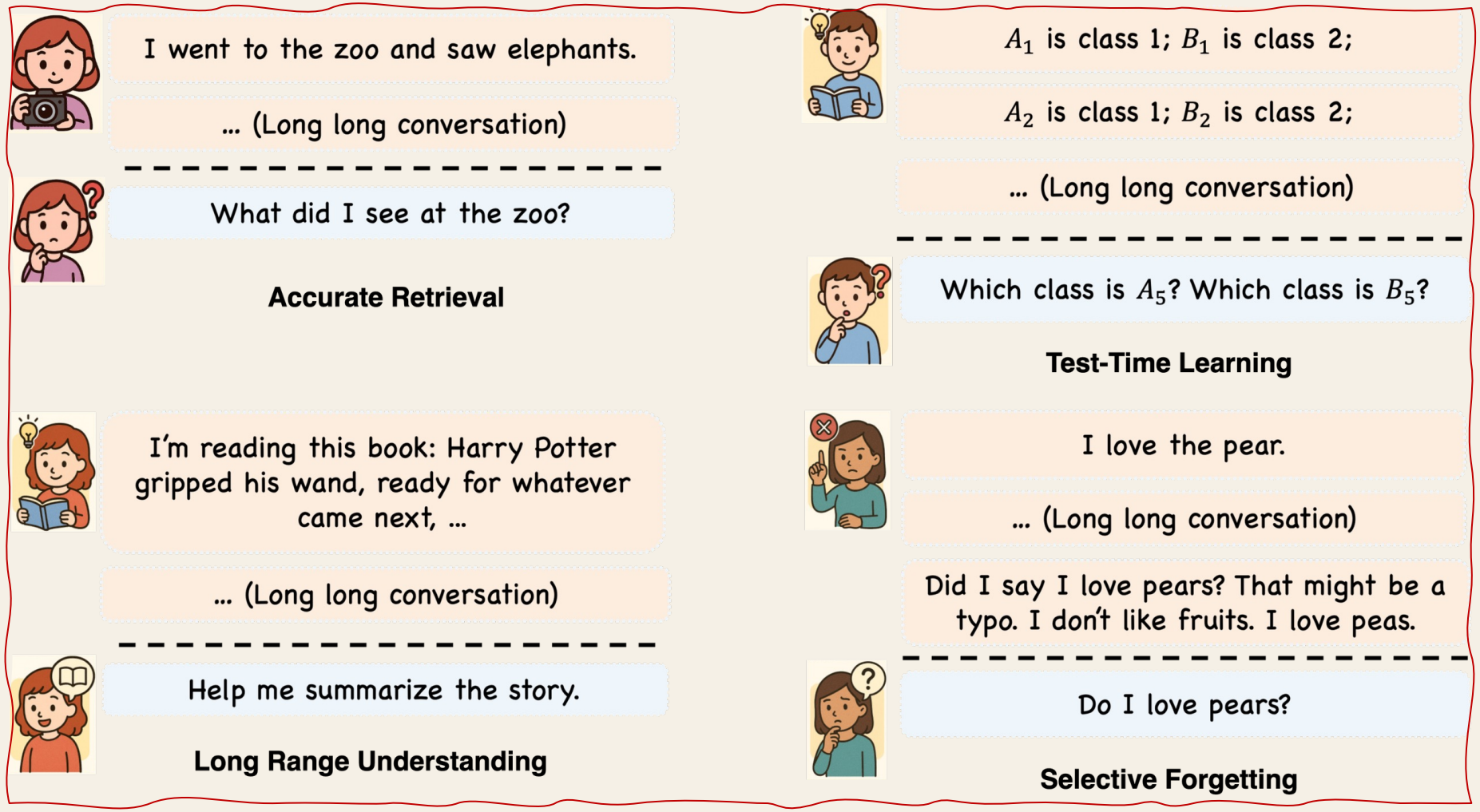


Figure 1: input above the dashed line; test question below. Hu et al., ICLR 2026.

Table 3: Overall Performance Comparison. In the absence of a specified model, All RAG agents and commercial memory agents use GPT-4o-mini as the backbone. Thus we highlight the performance of GPT-4o-mini as the reference. FC-SH and FC-MH mean FactConsolidation Single Hop and FactConsolidation Multi Hop, respectively. Best viewed in colors.

	AR					TTL			LRU			SF			Overall
Agent Type	SH-QA	MH-QA	LME(S*)	EventQA	**Avg.**	MCC	Recom.	**Avg.**	Summ.	DetQA	**Avg.**	FC-SH	FC-MH	**Avg.**	Scores
Long-Context Agents															
GPT-4o (128K)	72.0	51.0	32.0	77.2	58.1	87.6	12.3	50.0	32.2	77.5	54.9	60.0	5.0	32.5	48.8
GPT-4o-mini (128K)	64.0	43.0	30.7	59.0	49.2	82.0	15.1	48.6	28.9	63.4	46.2	45.0	5.0	25.0	42.2
Claude-3.7-Sonnet (200K)	77.0	53.0	34.0	74.6	59.7	**89.4**	**18.3**	**53.9**	52.5	71.8	62.2	43.0	2.0	22.5	49.6
GPT-5-mini (400K)	85.0	**71.0**	**63.3**	**78.2**	**74.4**	84.0	13.2	48.6	**56.3**	**76.1**	**66.2**	78.0	28.0	**53.0**	**60.6**
GPT-4.1-mini (1M)	83.0	66.0	55.7	82.6	71.8	75.6	16.7	46.2	41.9	56.3	49.1	36.0	5.0	20.5	46.9
Gemini-2.0-Flash (1M)	**87.0**	59.0	47.0	67.2	65.1	84.0	8.7	46.4	23.9	59.2	41.6	30.0	3.0	16.5	42.4
GPT-4o-mini	64.0	43.0	30.7	59.0	49.2	82.0	15.1	48.6	28.9	63.4	46.2	45.0	5.0	25.0	42.3
Simple RAG Agents															
BM25	66.0	56.0	45.3	**74.6**	60.5	75.4	13.6	44.5	**19.0**	52.1	35.6	48.0	3.0	25.5	41.5
Embedding RAG Agents															
Contriever	22.0	31.0	15.7	66.8	33.9	70.6	15.2	42.9	17.2	42.3	29.8	18.0	**7.0**	12.5	29.8
Text-Embed-3-Small	60.0	44.0	48.3	63.0	53.8	70.0	15.3	42.7	17.7	54.9	36.3	28.0	3.0	15.5	37.1
Text-Embed-3-Large	54.0	44.0	50.3	70.0	54.6	72.4	**16.2**	44.3	18.2	56.3	37.3	28.0	4.0	16.0	38.0
Qwen3-Embedding-4B	57.0	47.0	43.3	71.4	54.7	**78.0**	12.2	**45.1**	14.8	59.2	37.0	29.0	3.0	16.0	38.2
Structure-Augmented RAG Agents															
RAPTOR	29.0	38.0	34.3	45.8	36.8	59.4	12.3	35.9	13.4	42.3	27.9	14.0	1.0	7.5	27.0
GraphRAG	47.0	47.0	35.0	34.4	40.9	39.8	9.8	24.8	0.4	39.4	19.9	14.0	2.0	8.0	23.4
MemoRAG	29.0	33.0	20.0	56.0	34.5	77.0	13.1	**45.1**	9.2	50.7	30.0	21.0	**7.0**	14.0	30.9
HippoRAG-v2	**76.0**	66.0	50.7	67.6	**65.1**	61.4	10.2	35.8	14.6	57.7	36.2	**54.0**	5.0	**29.5**	**41.6**
Mem0	25.0	32.0	36.0	37.5	32.6	32.4	10.0	21.2	4.8	36.6	20.7	18.0	2.0	10.0	21.1
Cognee	31.0	26.0	29.3	26.8	28.3	35.4	10.1	22.8	2.3	29.6	16.0	28.0	3.0	15.5	20.6
Zep	44.0	25.0	38.3	42.5	37.5	62.8	12.1	37.5	4.2	28.2	16.2	7.0	3.0	5.0	24.0
Agentic Memory Agents															
Self-RAG	35.0	42.0	25.7	31.8	33.6	11.6	12.8	12.2	0.9	35.2	18.1	19.0	3.0	11.0	18.7
MemGPT	41.0	38.0	32.0	26.2	34.3	67.6	14.0	40.8	2.5	42.3	22.4	28.0	3.0	15.5	28.3
MIRIX	62.0	61.0	37.3	29.8	47.5	38.4	9.8	24.1	9.9	40.8	25.4	14.0	2.0	8.0	26.2
MIRIX (4.1-mini)	73.0	**75.0**	**51.0**	53.0	63.0	61.0	10.3	35.7	18.9	**62.0**	**40.5**	20.0	3.0	11.5	37.7

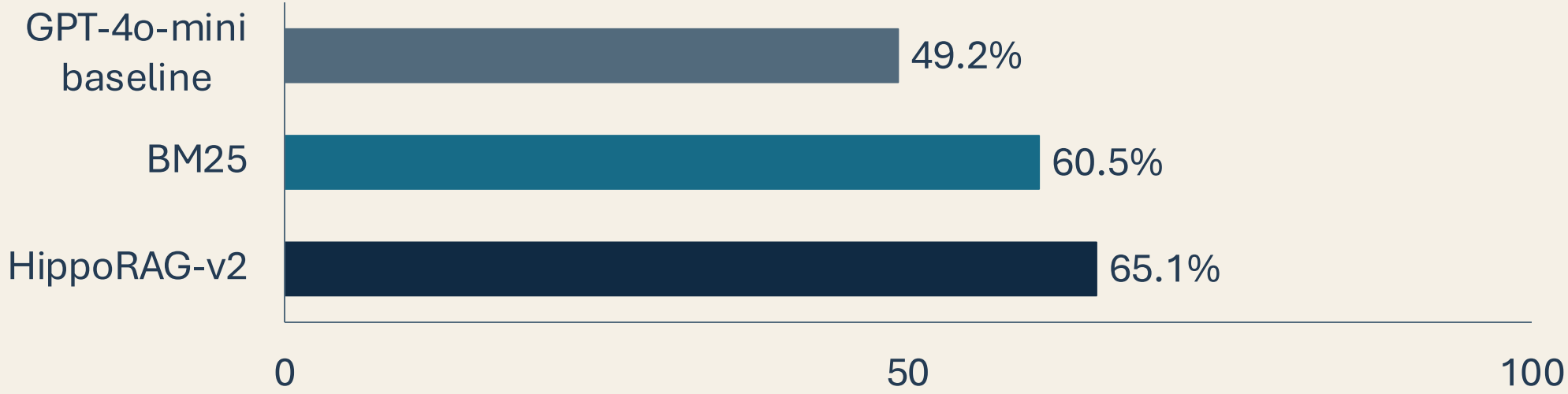
Compare each method across the four memory abilities.

Complete original Table 3: Hu, Wang, and McAuley, ICLR 2026, p. 8.

Retrieval can improve factual recall

All three methods use GPT-4o-mini. The baseline receives the full history.

Average accuracy across four factual-retrieval tasks (%).



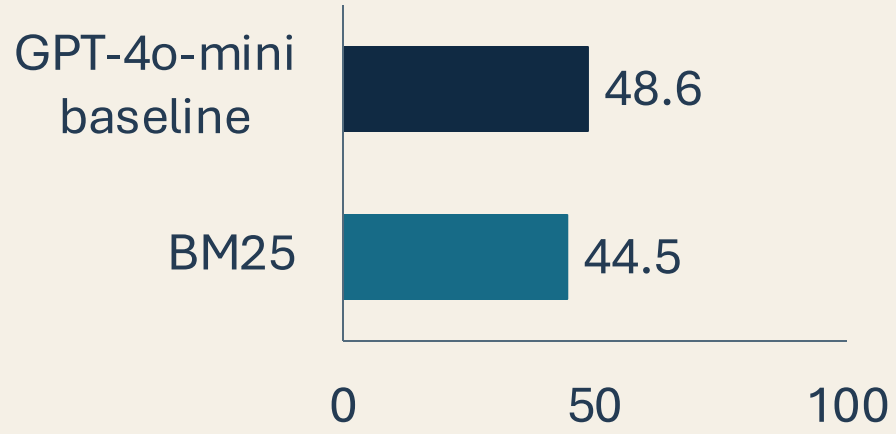
BM25 and HippoRAG-v2 scored higher than the baseline.
Other retrieval methods in Table 3 scored lower.

Retrieval accuracy and answer model: Hu et al., Table 3 and §4.1.

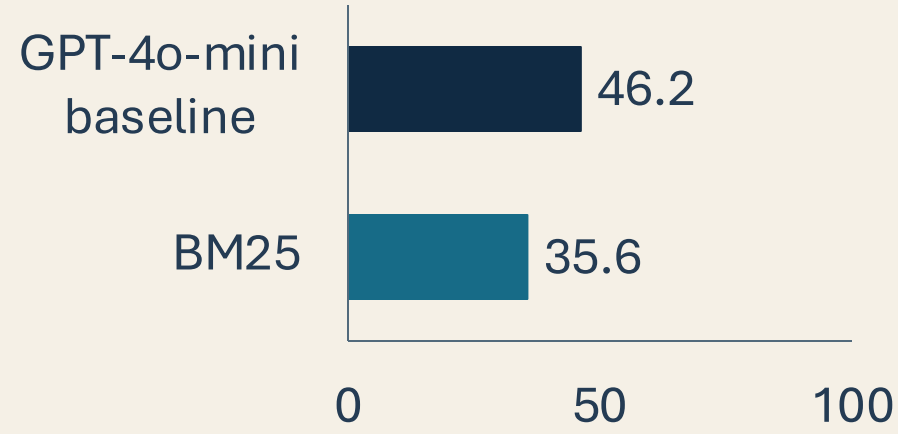
Learning and understanding: GPT-4o-mini baseline versus BM25

Both use GPT-4o-mini. The baseline receives the full history.
BM25 retrieves selected passages for the model.

Test-time learning



Long-range understanding



These tasks use examples or information spread across a long history.
Retrieval can leave out material the answer needs.

Average scores and their interpretation: Hu et al., Table 3 and §4.2.

Longer histories reduced correction accuracy

FactConsolidation tests whether answers use updated facts.
Multi-hop questions require combining several facts.
The best multi-hop accuracy in Table 3 was 28%.
Table 5: accuracy with 6,000 versus 32,000 history tokens.

Model	6K-token history	32K-token history
GPT-4o	28.0%	10.0%
o4-mini	80.0%	14.0%

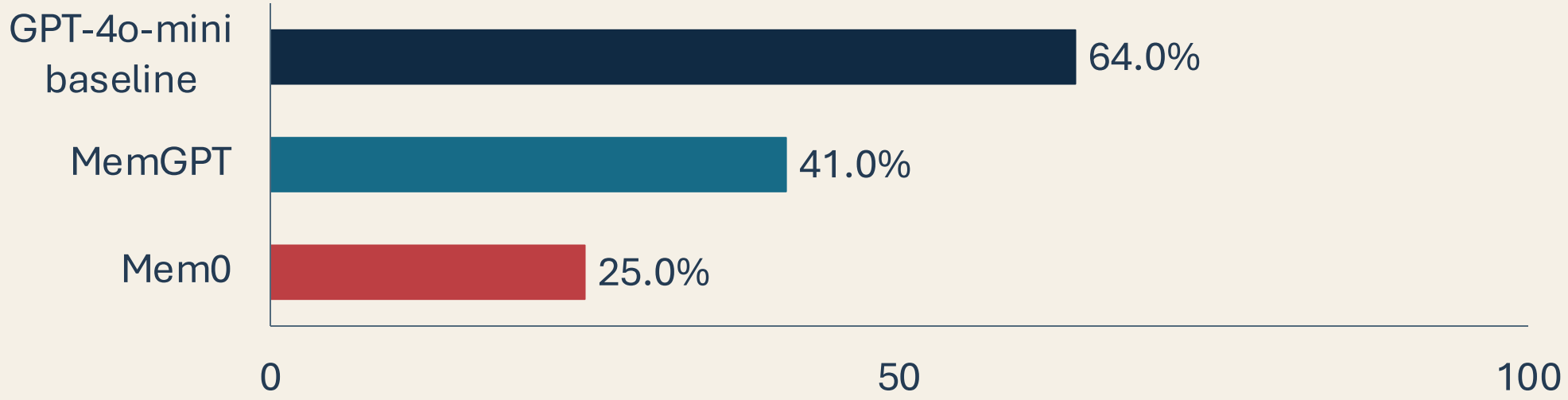
Both models were less accurate with 32,000 history tokens.
Check the stored records separately to verify deletion.

Correction accuracy: Hu et al., Tables 3 and 5.

MemGPT and Mem0 scored below the baseline

Answer accuracy on questions about one fact from a document (%).

All three methods use GPT-4o-mini. The baseline receives the full history.

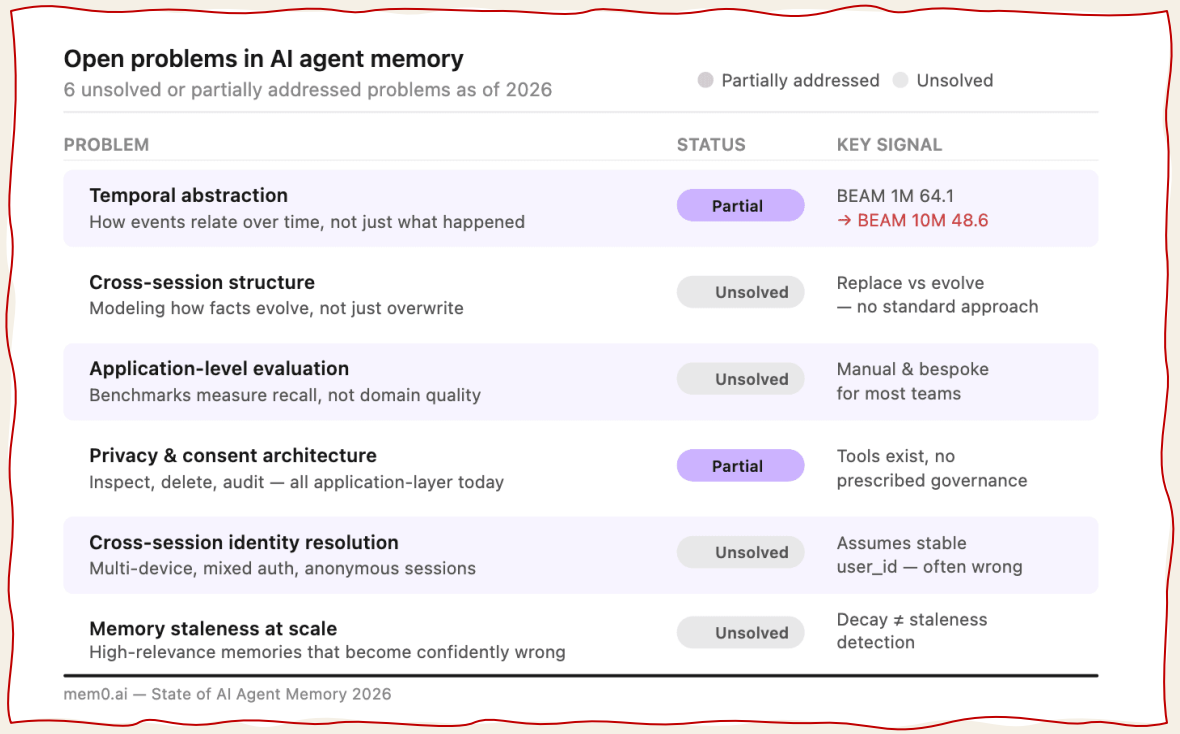


These scores describe the systems as configured in the paper.
Storage, retrieval, and chunk sizes differ across these configurations.

Single-hop accuracy: Hu et al., Table 3. Tested settings: Appendix F.

Open problems in AI agent memory

Mem0 identifies six problems that remain partly solved or unsolved.



Open problems in AI agent memory

6 unsolved or partially addressed problems as of 2026

Partially addressed · Unsolved

PROBLEM	STATUS	KEY SIGNAL
Temporal abstraction How events relate over time, not just what happened	Partial	BEAM 1M 64.1 → BEAM 10M 48.6
Cross-session structure Modeling how facts evolve, not just overwrite	Unsolved	Replace vs evolve — no standard approach
Application-level evaluation Benchmarks measure recall, not domain quality	Unsolved	Manual & bespoke for most teams
Privacy & consent architecture Inspect, delete, audit — all application-layer today	Partial	Tools exist, no prescribed governance
Cross-session identity resolution Multi-device, mixed auth, anonymous sessions	Unsolved	Assumes stable user_id — often wrong
Memory staleness at scale High-relevance memories that become confidently wrong	Unsolved	Decay ≠ staleness detection

mem0.ai — State of AI Agent Memory 2026

- **Temporal abstraction:** Reason about the order and timing of events across a long history.
- **Cross-session structure:** Track earlier and current facts, including when they changed.
- **Application-level evaluation:** Test whether memory improves real tasks.
- **Privacy and consent:** Let users inspect and delete memories.
- **Cross-session identity:** Identify a user across devices.
- **Memory staleness:** Detect when a stored fact becomes false.

Six open problems and original figure: Mem0, State of AI Agent Memory 2026.

Choosing what carries into the next call or task

The weekend planner needs different saved information for its next call, a crash, and a new task.

- **Current model call:** provide the request and useful retrieved evidence.
- **Interrupted task:** restore checkpoint state and pending work.
- **Next weekend:** recall relevant visits and confirmed preferences.
- **New evidence:** correct mistakes, merge duplicates, and remove outdated memories.
- **Verification:** test replies, recovery behavior, and the stored records.

These changes affect application data. They do not require training model weights.

Research sources

Four core research papers, plus Hindsight as a supplemental system example.

Source	Contribution and source identity
Park et al., 2023	Observation, retrieval, reflection Generative Agents: arXiv 2304.03442
Packer et al., 2023/2024	Context/storage hierarchy MemGPT: arXiv 2310.08560
Chhikara et al., 2025	Memory extraction and updates Mem0: arXiv 2504.19413
Hu et al., ICLR 2026	Memory evaluation MemoryAgentBench: arXiv 2507.05257

Compare the experimental settings before carrying a result into another application.

Latimer et al., ACL 2026: Hindsight, retain, recall, and reflect.

Tutorial and execution references

The fifth reading is a tutorial. LangGraph documentation explains the execution and storage APIs.

- Emre Okcular, OpenAI Cookbook, Jan 5, 2026: Context Engineering for Personalization.
- **LangGraph checkpointers:** saved state, durability, and recovery.
- **LangGraph interrupts and stores:** human input and memory across threads.

Personalization: OpenAI tutorial. Persistence and interrupts: LangGraph docs.